Real-time Graphics Lab Book

Contents

Lab 1

Date: 02/10/2025

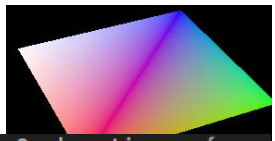
Exercise 1:

Question:

Draw Two Triangles Without Using Vertex Indices
Render two distinct triangles instead of one quad using vkCmdDraw().

Solution:

The project shows a 2D square rotating around its centre to start, using the vertex vector



Where the first brackets give the x,y,z position, and the second give the rgb value.

```
const std::vector<Vertex> Quad_vertices = {
    {{-0.5f, -0.5f, 0.0f}, {1.0f, 0.0f, 0.0f}},
    {{0.5f, -0.5f, 0.0f}, {0.0f, 1.0f, 0.0f}},
    {{0.5f, 0.5f, 0.0f}, {0.0f, 0.0f, 1.0f}},
    {{-0.5f, 0.5f, 0.0f}, {1.0f, 1.0f, 1.0f}}
};
```

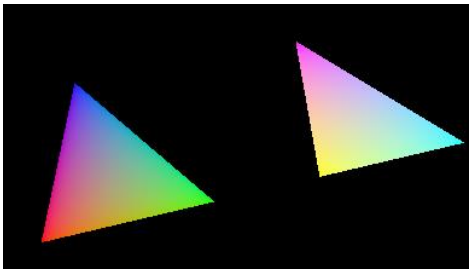
In order to draw 2 triangles, there must be 6 verticies.

```
const std::vector<Vertex> Quad_vertices = {
    // Triangle 1
    {{-0.8f, -0.5f, 0.0f}, {1.0f, 0.0f, 0.0f}},
    {{-0.2f, -0.5f, 0.0f}, {0.0f, 1.0f, 0.0f}},
    {{-0.5f,  0.4f, 0.0f}, {0.0f, 0.0f, 1.0f}},
    // Triangle 2
    {{ 0.2f, -0.5f, 0.0f}, {1.0f, 1.0f, 0.0f}},
    {{ 0.8f, -0.5f, 0.0f}, {0.0f, 1.0f, 1.0f}},
    {{ 0.5f,  0.4f, 0.0f}, {1.0f, 0.0f, 1.0f}}
};
```

The vkCmdDrawIndexed() function is replaced with ckCmdDraw(), with an increased buffer size to 6.

```
//vkCmdDrawIndexed(commandBuffer, static_cast<uint16_t>(indices.size()), 1, 0, 0, 0);
vkCmdDraw(commandBuffer, 6, 1, 0, 0);
```

Sample Output:



Reflection:

By manually defining six vertices with unique positions and colours, and adjusting the vertex buffer and draw call, I successfully displayed two separate coloured triangles on screen. This furthered my understanding of Vulkan's non-indexed rendering and memory allocation for vertex data.

Exercise 2:

Question:

Draw Two Squares Using An Index Buffer

Solution:

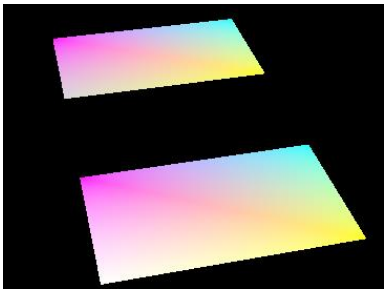
The vertices vector now defines 2 squares with 8 unique vertices. The indices define two triangles that share a squares' vertices.

```
const std::vector<Vertex> Quad_vertices = {
    // Square 1 (centered left)
    {{-0.9f, -0.5f, 0.0f}, {1,1,0}},
    {{-0.3f, -0.5f, 0.0f}, {0,1,1}},
    {{-0.3f,  0.5f, 0.0f}, {1,0,1}},
    {{-0.9f,  0.5f, 0.0f}, {1,1,1}},
    // Square 2 (centered right)
    {{ 0.3f, -0.5f, 0.0f}, {1,1,0}},
    {{ 0.9f, -0.5f, 0.0f}, {0,1,1}},
    {{ 0.9f,  0.5f, 0.0f}, {1,0,1}},
    {{ 0.3f,  0.5f, 0.0f}, {0.7f,0.7f,0.7f}}
};

const std::vector<uint16_t> Quad_indices = {
    // First square
    0,1,2, 2,3,0,
    // Second square
    4,5,6, 6,7,4
};
```

An index count of 12 is used.

```
vkCmdDrawIndexed(commandBuffer, static_cast<uint16_t>(indices.size()), 1, 0, 0, 0);
```

Sample Output:



Reflection:

This demonstrated how an index buffer can be used to efficiently render squares by reusing shared vertex data. By defining four unique corner vertices and creating an index vector to from two triangles per square, reducing redundancy. This approach improved memory usage and demonstrated how vkCmdDrawIndexed and vkCmdBindIndexBuffer work together in Vulkan's rendering pipeline.

Exercise 3, 4, 5:

Question:

Extend the previous exercise to draw the four side faces of a cube

Solution:

8 cube vertices are now included, added 25 indices for the 4 lateral faces.

```
const std::vector<Vertex> Quad_vertices = {
    // Front face (z = +0.5)
    {{-0.5f, -0.5f,  0.5f}, {1.0f, 1.0f, 1.0f}}, // v0
    {{ 0.5f, -0.5f,  0.5f}, {1.0f, 1.0f, 0.0f}}, // v1
    {{ 0.5f,  0.5f,  0.5f}, {1.0f, 1.0f, 1.0f}}, // v2
    {{-0.5f,  0.5f,  0.5f}, {0.0f, 1.0f, 1.0f}}, // v3
    // Back face (z = -0.5)
    {{-0.5f, -0.5f, -0.5f}, {1.0f, 0.0f, 1.0f}}, // v4
    {{ 0.5f, -0.5f, -0.5f}, {1.0f, 1.0f, 1.0f}}, // v5
    {{ 0.5f,  0.5f, -0.5f}, {0.0f, 1.0f, 0.0f}}, // v6
    {{-0.5f,  0.5f, -0.5f}, {1.0f, 1.0f, 1.0f}}  // v7
};

const std::vector<uint16_t> Quad_indices = {
    // Front
    0,1,2,  2,3,0,
    // Right
    1,5,6,  6,2,1,
    // Back
    5,4,7,  7,6,5,
    // Left
    4,0,3,  3,7,4
};
```

Disabling Cull mode ensures all side triangles remain visible during rotation.

```
rasterizer.cullMode = VK_CULL_MODE_NONE;
```

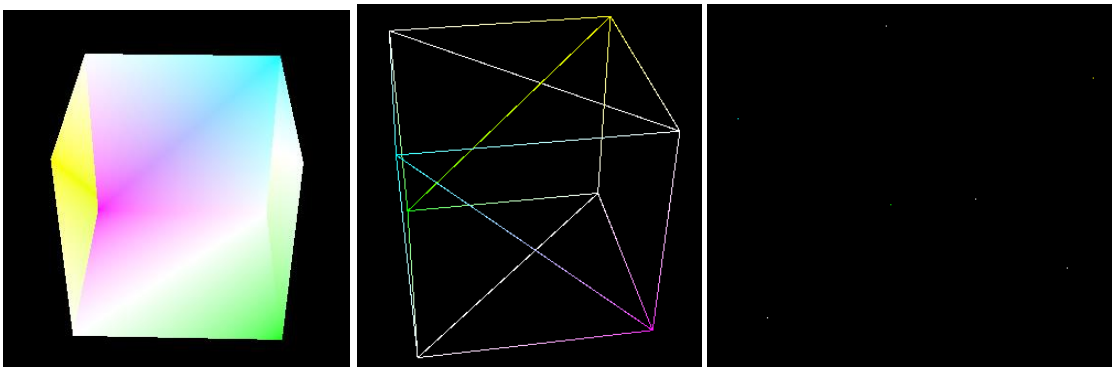
Changing from FILL -> LINE mode displays the cube as a wireframe

```
rasterizer.polygonMode = VK_POLYGON_MODE_LINE;
```

Changing from TRIANGLE -> POINT list displays only the vertices

```
inputAssembly.topology = VK_PRIMITIVE_TOPOLOGY_POINT_LIST;
```

Sample Output:



Reflection:

In Exercise 3, I extended the cube rendering by defining 8 vertices and 24 indices to draw its four side walls, and disabled back-face culling to ensure all faces were visible during rotation. Exercise 4 involved switching the polygon mode to VK_POLYGON_MODE_LINE, allowing the cube to be rendered as a wireframe that highlights its edges. In Exercise 5, I created a new pipeline with point topology to render only the cube's 8 vertices as distinct points, showcasing their spatial positions.

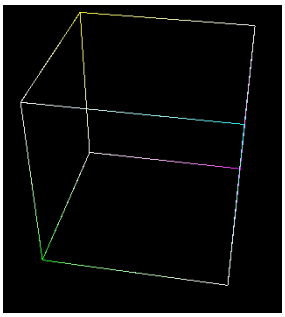
Exercise 6:

Question:

RENDER THE CUBE'S EDGES AS LINES

Solution:

Changing from TRIANGLE -> LINE list displays only edges.



```
inputAssembly.topology = VK_PRIMITIVE_TOPOLOGY_LINE_LIST;
```

```
const std::vector<uint16_t> Quad_indices = {
    // 12 cube edges (line list: 24 indices)
    0,1,  1,2,  2,3,  3,0,        // Front face perimeter
    4,5,  5,6,  6,7,  7,4,        // Back face perimeter
    0,4,  1,5,  2,6,  3,7         // Side edges connecting front/back
};
```

Reflection:

This exercise involved rendering the cube's 12 edges using line segments by creating an index buffer with 24 indices, each pair representing one edge. A new graphics pipeline was configured with VK_PRIMITIVE_TOPOLOGY_LINE_LIST to interpret the vertex data as lines rather than filled polygons. The result was a wireframe cube composed of individual line segments, clearly outlining its structure.

Exercise 7:

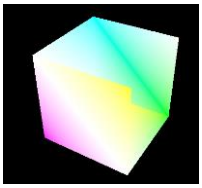
Question:

Render the cube using the VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP topology.

Solution:

Replaced 24 line indices with a 14-index triangle strip, covering all 6 faces.

```
const std::vector<uint16_t> Quad_indices = {
    // Minimal cube triangle strip (re-uses shared vertices)
    // Produces 12 triangles: (n - 2) with n = 14
    0, 1, 3, 2, 6, 1, 5, 0, 4, 3, 7, 6, 4, 5
```



Changing from LINE -> FILL to render solid triangles

```
rasterizer.polygonMode = VK_POLYGON_MODE_FILL;
```

New index buffer including a full triangle list with 36 points.

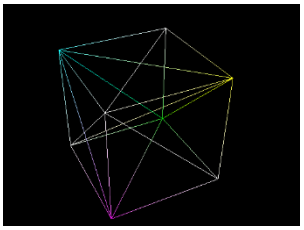
```
const std::vector<uint16_t> Quad_indices = {
    // Front
    0, 1, 2,  2, 3, 0,
    // Back
    4, 7, 6,  6, 5, 4,
    // Left
    0, 4, 7,  7, 3, 0,
    // Right
    1, 5, 6,  6, 2, 1,
    // Top
    3, 7, 6,  6, 2, 3,
    // Bottom
    0, 4, 5,  5, 1, 0
};
```

```
rasterizer.polygonMode = VK_POLYGON_MODE_LINE;
```

Changing from LIST -> Strip builds a chain of connected triangles

```
inputAssembly.topology = VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP;
```

Reflection:



This exercise focused on rendering a cube using the TRIANGLE_STRIP topology to improve efficiency by reducing the number of vertices needed. An ordered index buffer was created, including degenerate triangles to seamlessly connect disjoint faces without breaking the strip. The result was a solid cube rendered with a single optimized draw call, demonstrating effective use of triangle strips in Vulkan.

Exercise 8:

Question:

DRAWING MULTIPLE CUBES USING INSTANCED DRAWING

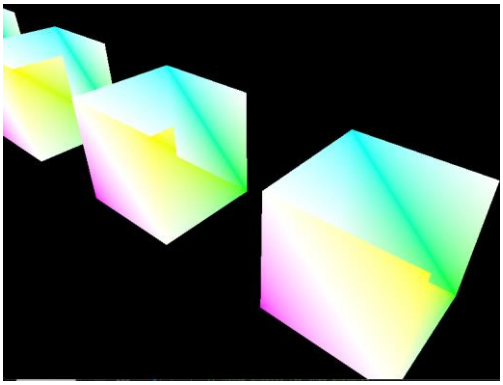
Solution:

Replaced single-cube load with batch duplication.
The code duplicates the original cube's vertices and indices INSTANCE_COUNT times, translating each copy along +X so they form a centered row.

For each instance it adds xOffset to the vertex positions, then remaps the indices by adding a baseVertex offset so they reference the correct new vertex block.

This produces one large vertex/index buffer so a single vkCmdDrawIndexed renders all cubes.

```cpp
void loadModel() {
    const size_t baseVertexCount = Quad_vertices.size();   // 8
    const size_t baseIndexCount = Quad_indices.size();     // 36

    vertices.clear();
    indices.clear();
    vertices.reserve(baseVertexCount * INSTANCE_COUNT);
    indices.reserve(baseIndexCount * INSTANCE_COUNT);

    // Center the row of cubes around the origin
    float centerShift = (INSTANCE_COUNT - 1) * 0.5f * INSTANCE_SPACING;

    for (uint32_t i = 0; i < INSTANCE_COUNT; ++i) {
        float xOffset = i * INSTANCE_SPACING - centerShift;

        // Duplicate & offset vertices
        for (const auto& v : Quad_vertices) {
            Vertex nv = v;
            nv.pos.x += xOffset;   // Only translate along X
            vertices.push_back(nv);
        }

        // Remap indices for this instance
        uint16_t baseOffset = static_cast<uint16_t>(i * baseVertexCount);
        for (auto idx : Quad_indices) {
            indices.push_back(static_cast<uint16_t>(idx + baseOffset));
        }
    }
}
```

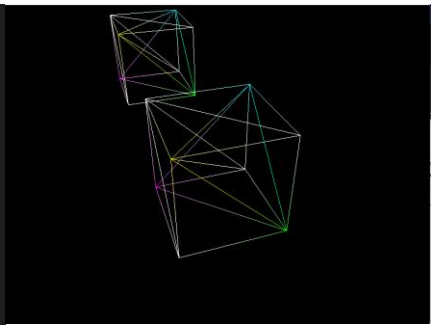


Shader Modification

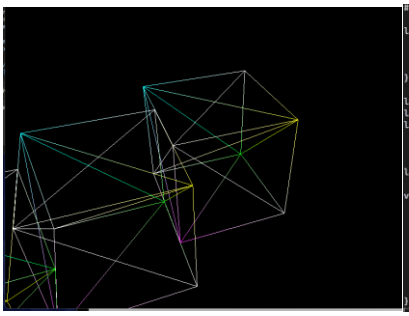
The shader takes a base model/view/projection from a uniform buffer and, for each instance, builds a translation along +X using gl_InstanceIndex (spacing 3.0 units, first instance centered by subtracting 3.0) and composes it with the model matrix.

It then transforms each vertex by proj * view * (model * per-instance translation) and passes the vertex color through.

```cpp
#version 450

layout(binding = 0) uniform UniformBufferObject {
    mat4 model;
    mat4 view;
    mat4 proj;
} ubo;

layout(location = 0) in vec3 inPosition;
layout(location = 1) in vec3 inColor;

layout(location = 0) out vec3 fragColor;


void main() {
    vec3 offset = vec3(gl_InstanceIndex * 3.0 - 3.0, 0.0, 0.0);
    mat4 translation = mat4(1.0);
    translation[3] = vec4(offset, 1.0);
    mat4 instanceModel = ubo.model * translation;

    gl_Position = ubo.proj * ubo.view * instanceModel * vec4(inPosition, 1.0);
    fragColor = inColor;
}
```



This solution creates separate instances of each cube, making them spin separately.



```cpp
#version 450

layout(binding = 0) uniform UniformBufferObject {
    mat4 model;
    mat4 view;
    mat4 proj;
} ubo;

layout(location = 0) in vec3 inPosition;
layout(location = 1) in vec3 inColor;
layout(location = 2) in mat4 instanceModel;


layout(location = 0) out vec3 fragColor;

void main() {
    mat4 instanceModel = ubo.model;

    // Offset by instance index, e.g., move instances along X axis
    float offset = float(gl_InstanceIndex) * 1.1;
    instanceModel[3] = vec4(offset, 0.0, 0.0, 1.0);

    gl_Position = ubo.proj * ubo.view * instanceModel * vec4(inPosition, 1.0);
    fragColor = inColor;
}
```

Reflection:

This exercise introduced instancing, a technique that allows the GPU to render multiple copies of the same mesh using a single draw call, significantly improving performance. By setting the instanceCount parameter in vkCmdDrawIndexed and using gl_InstanceIndex in the vertex shader, each instance can be uniquely positioned without updating the uniform buffer. This method is ideal for rendering large numbers of identical objects efficiently, reducing CPU overhead and draw call complexity.

Exercise 9:

Question:

DRAWING TWO CUBES USING PUSH CONSTANTS

Solution:

The model matrix is moved out of the CPU struct UniformBufferObject

```
struct UniformBufferObject {
    alignas(16) glm::mat4 view;
    alignas(16) glm::mat4 proj;
};

struct ModelPushConstant {
    glm::mat4 model;
};
```

The pipeline layout now includes a VkPushConstantRange for VK_SHAER_STAGE_VERTEX_BIT

```
VkPushConstantRange pushConstantRange{};
pushConstantRange.stageFlags = VK_SHADER_STAGE_VERTEX_BIT;
pushConstantRange.offset = 0;
pushConstantRange.size = sizeof(ModelPushConstant);

VkPipelineLayoutCreateInfo pipelineLayoutInfo{};
pipelineLayoutInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO;
pipelineLayoutInfo.setLayoutCount = 1;  // Number of descriptor sets
pipelineLayoutInfo.pSetLayouts = &descriptorSetLayout;
pipelineLayoutInfo.pushConstantRangeCount = 1;  //
pipelineLayoutInfo.pPushConstantRanges = &pushConstantRange; //
```

recordCommandBuffer has two different model matrices pushed with vkCmdPushConstants, before each DrawIndexed.

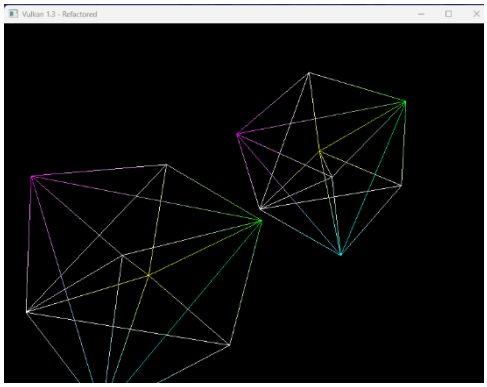
```
vkCmdBindDescriptorSets(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, pipelineLayout, 0, 1, &descriptorSets[currentFrame], 0, nullptr);

VkBuffer vertexBuffers[] = { vertexBuffer };
VkDeviceSize offsets[] = { 0 };
vkCmdBindVertexBuffers(commandBuffer, 0, 1, vertexBuffers, offsets);
vkCmdBindIndexBuffer(commandBuffer, indexBuffer, 0, VK_INDEX_TYPE_UINT16);
```

The vertex shader should read view/proj from the UBO and model from the push constant, computing gl_Position = ubo.proj * ubo.view * pc.model * vec4(inPosition, 1.0), which yields two side-by-side cubes without duplicating geometry or descriptor sets.

```
// Cube 1: rotate around Y and translate left
pushConstant.model = glm::translate(glm::mat4(1.0f), glm::vec3(-1.0f, 0.0f, 0.0f));
pushConstant.model = glm::rotate(pushConstant.model, time, glm::vec3(0.0f, 1.0f, 0.0f));
vkCmdPushConstants(commandBuffer, pipelineLayout, VK_SHADER_STAGE_VERTEX_BIT, 0, sizeof(ModelPushConstant), &pushConstant);
vkCmdDrawIndexed(commandBuffer, static_cast<uint32_t>(indices.size()), 1, 0, 0, 0);

// Cube 2: rotate around Y opposite direction and translate right
pushConstant.model = glm::translate(glm::mat4(1.0f), glm::vec3(1.0f, 0.0f, 0.0f));
pushConstant.model = glm::rotate(pushConstant.model, -time, glm::vec3(0.0f, 1.0f, 0.0f));
vkCmdPushConstants(commandBuffer, pipelineLayout, VK_SHADER_STAGE_VERTEX_BIT, 0, sizeof(ModelPushConstant), &pushConstant);
vkCmdDrawIndexed(commandBuffer, static_cast<uint32_t>(indices.size()), 1, 0, 0, 0);
```

Reflection:

This exercise demonstrated how to render two wireframe cubes at different positions using push constants to efficiently pass transformation data to the vertex shader. Instead of duplicating vertex data or updating uniform buffers, each draw call pushed a unique model matrix to reposition the cube. This method allows for fast, per-object updates during command buffer recording and is ideal for rendering multiple instances of the same mesh with distinct transformations.



Lab 2

Date: 09/10/25

Exercise 1:

Question:

CREATE A FLAT GRID

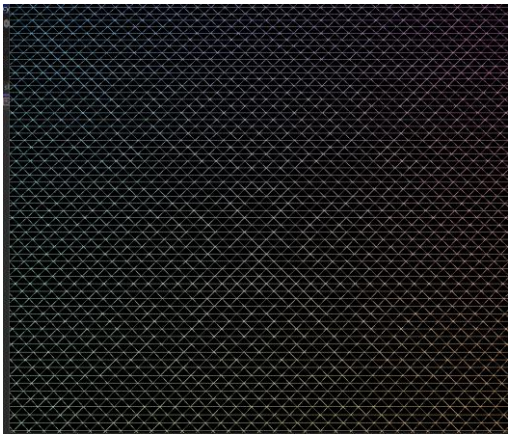
Solution:

createGrid() clears and fills outVertices with a (cellsX+1)×(cellsZ+1) lattice of positions on the XZ plane (y=0), centred around the origin with spacing cellSize, and assigns UV-based colours per vertex.

It then builds outIndices with 6 indices per cell (two triangles) using a row-major index lambda to form a triangle-list grid.

```
createGrid(100, 100, 0.1f, vertices, indices);
```

Sample Output:



```
GeometryGenerator::MeshData GeometryGenerator::CreateGrid(
    uint32_t cellsX, uint32_t cellsZ, float cellSize)
{
    MeshData meshData;

    const uint32_t vertCountX = cellsX + 1;
    const uint32_t vertCountZ = cellsZ + 1;
    meshData.vertices.reserve(vertCountX * vertCountZ);
    meshData.indices.reserve(cellsX * cellsZ * 6);

    const float totalWidth = cellsX * cellSize;
    const float totalDepth = cellsZ * cellSize;
    const float originX = -totalWidth * 0.5f;
    const float originZ = -totalDepth * 0.5f;

    for (uint32_t z = 0; z < vertCountZ; ++z) {
        for (uint32_t x = 0; x < vertCountX; ++x) {
            float px = originX + x * cellSize;
            float pz = originZ + z * cellSize;

            float u = static_cast<float>(x) / static_cast<float>(cellsX);
            float v = static_cast<float>(z) / static_cast<float>(cellsZ);

            Vertex vtx{};
            vtx.pos = { px, 0.0f, pz };
            vtx.color = { u, v, 1.0f - u };
            meshData.vertices.push_back(vtx);
        }
    }

    auto idx = [vertCountX](uint32_t x, uint32_t z) {
        return z * vertCountX + x;
    };

    for (uint32_t z = 0; z < cellsZ; ++z) {
        for (uint32_t x = 0; x < cellsX; ++x) {
            uint32_t v0 = idx(x, z);
            uint32_t v1 = idx(x + 1, z);
            uint32_t v2 = idx(x, z + 1);
            uint32_t v3 = idx(x + 1, z + 1);

            meshData.indices.push_back(v0);
            meshData.indices.push_back(v2);
            meshData.indices.push_back(v1);
            meshData.indices.push_back(v2);
            meshData.indices.push_back(v3);
            meshData.indices.push_back(v1);
        }
    }

    return meshData;
```

Reflection:

The implementation involves translating grid dimensions into concrete vertex position in 3D space by iterating through rows and columns, calculating positions relative to the origin.

Exercise 2:

Question:

CREATE A WAVY TERRAIN

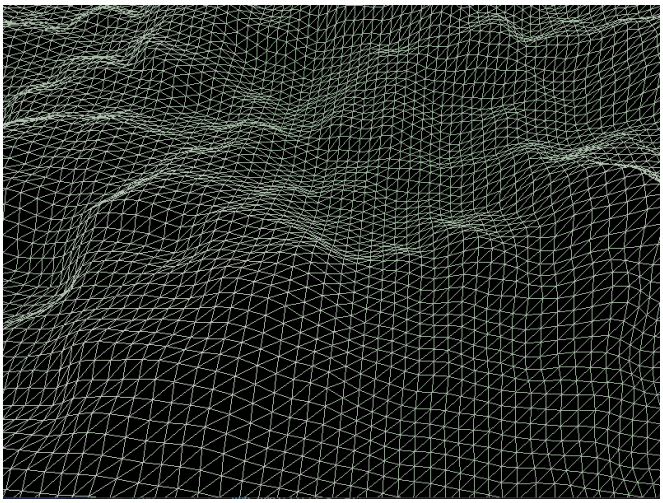
Solution:

createTerrain() builds a height-mapped terrain mesh on an (cellsX+1) by (cellsZ+1) lattice in the XZ plane with spacing cellSize, centered at the origin, clearing and reserving the output buffers first.

For each vertex, it computes y from Perlin-like fractal noise (baseFrequency, octaves, persistence, lacunarity), scales it by heightScale, and assigns a color blended between low and high based on normalized height.

It then emits indices for two triangles per grid cell (row-major), populating outIndices for a triangle-list.

Sample Output:



Reflection:

This exercise demonstrated how procedural height-field generation using octave-based Perlin noise can create realistic terrain topology while maintaining a simple grid-based mesh structure, with the key insight being that complex natural surfaces emerge from layering multiple frequencies of smooth noise rather than from complex geometry.

Exercise 3:

Question:

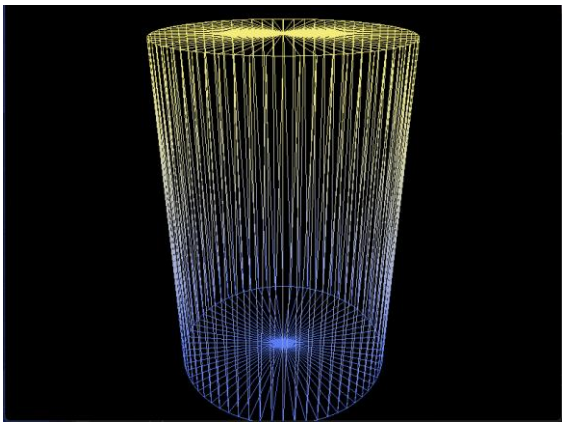
PROCEDURAL CYLINDER

Solution:

createCylinder builds two vertex rings at y = ±height/2 plus one center vertex per cap, using x = R·cos(theta), z = R·sin(theta) sampled over “segments” (clamped to ≥3), and reserves capacity for efficiency.

It emits indices per segment for: a bottom fan (center, next, current), a top fan (center, current, next), and two side-wall triangles linking bottom and top rings, producing a coherent triangle list with consistent winding.

Sample Output:



```
//auto mesh = GeometryGenerator::CreateCylinder(0.5f, 1.5f, 64);
 auto mesh = GeometryGenerator::CreateSphere(0.5f, 32, 16);
// auto mesh = GeometryGenerator::CreateGrid(100, 100, 0.1f);
// auto mesh = GeometryGenerator::CreateTerrain(100, 100, 0.2f, 5.0f, 0.1f);


vertices = mesh.vertices;
indices = mesh.indices;
```

Reflection:

This exercise reinforced the principle of constructive solid geometry by decomposing a cylinder into parametric circular cross-sections and demonstrating how triangle fans (for endcaps) and quad strips (for sidewalls) can be efficiently indexed from a minimal vertex set, highlighting the importance of consistent winding order for proper backface culling in wireframe rendering.

Exercise 4:

Question:

CREATING A REUSABLE GEOMETRY UTILITY

Solution:

GeometryGenerator defines a nested MeshData struct containing std::vector<Vertex> for vertex data and std::vector<uint32_t> for index data.

The public interface exposes four static factory methods: CreateGrid for flat XZ-plane grids, CreateCylinder for capped cylinders, CreateSphere for UV-sphere tessellation, and CreateTerrain for heightfield generation with optional Perlin noise.

Private members include Perlin noise implementation helpers (Noise2D, Fade, Grad) and a 256-element permutation table P used for pseudo-random gradient hashing in fractal terrain synthesis.

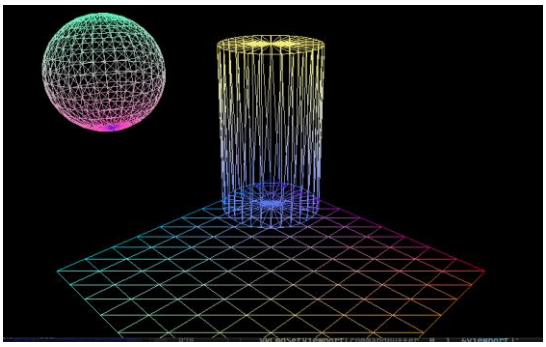
The DrawCommand struct enables per-object rendering.

```
struct DrawCommand {
    uint32_t indexCount;
    uint32_t firstIndex;
    int32_t vertexOffset;
    glm::mat4 modelMatrix;
};
std::vector<DrawCommand> drawCommands;
```

```
vertexOffset = vertices.size();
indexOffset = indices.size();
vertices.insert(vertices.end(), grid.vertices.begin(), grid.vertices.end());
indices.insert(indices.end(), grid.indices.begin(), grid.indices.end());
drawCommands.push_back({
    static_cast<uint32_t>(grid.indices.size()),
    indexOffset,
    static_cast<int32_t>(vertexOffset),
    glm::translate(glm::mat4(1.0f), glm::vec3(0.0f, -1.0f, 0.0f))
    });
```

It can then be called within initVulkan()

```
auto sphere = GeometryGenerator::CreateSphere(0.5f, 32, 16);
auto cylinder = GeometryGenerator::CreateCylinder(0.3f, 1.0f, 32);
auto grid = GeometryGenerator::CreateGrid(10, 10, 0.2f);

vertices.clear();
indices.clear();
drawCommands.clear();
```



Reflection:

This exercise encapsulates procedural geometry generation into a reusable utility class with static factory methods that return self-contained MeshData structures, enabling clean separation of concerns between geometry generation and Vulkan rendering logic while facilitating scene composition through the DrawCommand abstraction that decouples mesh definitions from per-instance transformation state.

Exercise 6:

Question:

LOADING EXTERNAL MODELS WITH ASSIMP

Solution:

The LoadModelFromFile function uses the Assimp library to load 3D model data from a file and converts it into a custom MeshData structure.

```
GeometryGenerator::MeshData GeometryGenerator::LoadModelFromFile(
    const std::string& filepath,
    const glm::vec3& defaultColor)
{
    MeshData meshData;
    Assimp::Importer importer;

    const aiScene* scene = importer.ReadFile(filepath,
        aiProcess_Triangulate |           // Convert all primitives to triangles
        aiProcess_FlipUVs |               // Flip UV coordinates (if needed)
        aiProcess_GenNormals |            // Generate normals if not present
        aiProcess_JoinIdenticalVertices); // Optimize by joining identical vertices

    if (!scene || scene->mFlags & AI_SCENE_FLAGS_INCOMPLETE || !scene->mRootNode) {
        throw std::runtime_error(std::string("Assimp Error: ") + importer.GetErrorString());
    }

    if (scene->mNumMeshes == 0) {
        throw std::runtime_error("No meshes found in model file: " + filepath);
    }
```

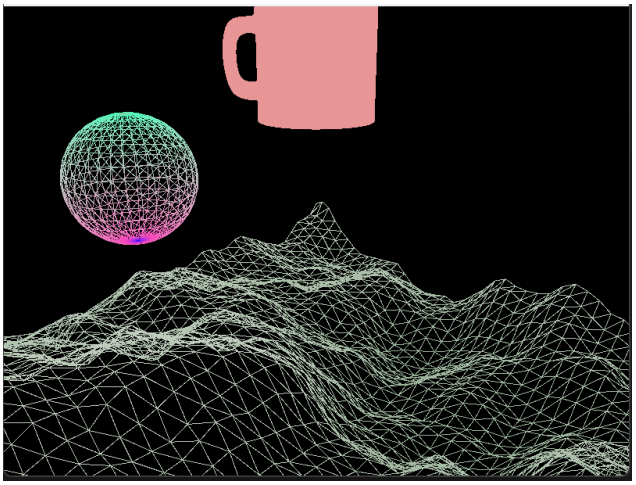
```
for (unsigned int meshIndex = 0; meshIndex < scene->mNumMeshes; ++meshIndex) {
    aiMesh* mesh = scene->mMeshes[meshIndex];

    uint32_t vertexOffset = static_cast<uint32_t>(meshData.vertices.size());

    meshData.vertices.reserve(meshData.vertices.size() + mesh->mNumVertices);
    meshData.indices.reserve(meshData.indices.size() + mesh->mNumFaces * 3);

    for (unsigned int i = 0; i < mesh->mNumVertices; i++) {
        Vertex vertex;

        vertex.pos = {
            mesh->mVertices[i].x,
            mesh->mVertices[i].y,
            mesh->mVertices[i].z
        };

        if (mesh->HasVertexColors(0)) {
            vertex.color = {
                mesh->mColors[0][i].r,
                mesh->mColors[0][i].g,
                mesh->mColors[0][i].b
            };
        }
        else {
            vertex.color = defaultColor;
        }

        meshData.vertices.push_back(vertex);
    }
```

It iterates through all meshes in the loaded scene, extracting vertex positions and colours (or using a default colour if none are present) while maintaining proper index offsets to combine multiple meshes into a single buffer.

The function applies several post-processing steps including triangulation and normal generation and throws exceptions if the file cannot be loaded or contains no mesh data.

Reflection:

This exercise demonstrated third-party library integration by incorporating Assimp's model loading capabilities through NuGet package management, where the LoadModelFromFile function parses industry-standard file formats (like OBJ) into application-specific vertex/index data structures by iterating through the Assimp scene graph, extracting positional and color attributes while applying post-processing flags that normalize mesh topology for efficient GPU rendering, thereby decoupling procedural geometry generation from asset-driven content pipelines.



Lab 3

Date: 14/10/2025

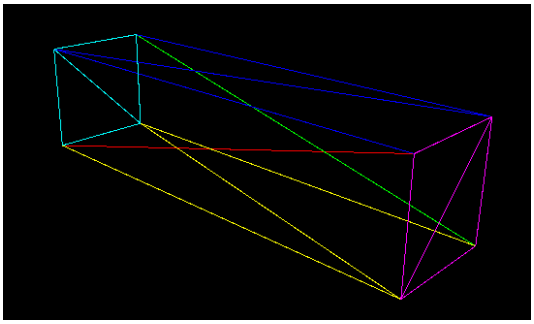
Exercise 1:

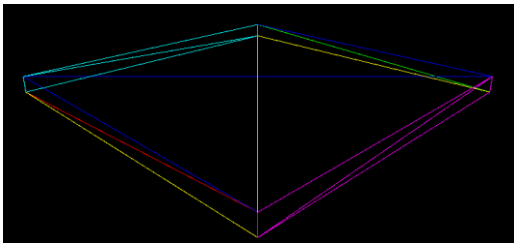
Question:

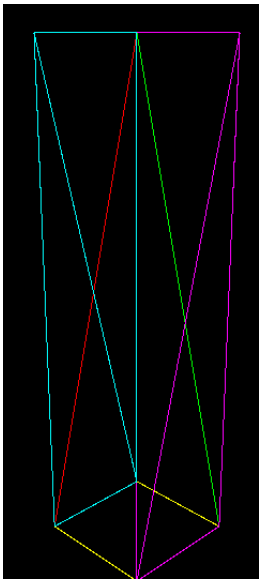
BASIC SCALING TRANSFORMATION

Solution:

```
auto cube = GeometryGenerator::CreateCube(1.0f, 1.0f, 1.0f);

vertexOffset = vertices.size();
indexOffset = indices.size();
vertices.insert(vertices.end(), cube.vertices.begin(), cube.vertices.end());
indices.insert(indices.end(), cube.indices.begin(), cube.indices.end());

glm::mat4 modelMatrix = glm::mat4(1.0f);

//glm::mat4 modelMatrix = glm::scale(glm::mat4(1.0f), glm::vec3(0.5f, 2.0f, 0.5f)); // stretch in Y

//glm::mat4 modelMatrix = glm::scale(glm::mat4(1.0f), glm::vec3(2.0f, 0.5f, 0.5f)); // stretch in X

//glm::mat4 modelMatrix = glm::scale(glm::mat4(1.0f), glm::vec3(2.0f, 0.1f, 2.0f)); // flat in Y

drawCommands.push_back({
    static_cast<uint32_t>(cube.indices.size()),
    indexOffset,
    static_cast<int32_t>(vertexOffset),
    modelMatrix,
    false,
    false
    });
```







Reflection:

The use of glm::scale() showed how increasing and decreasing values in the x,z and z axis can manipulate shapes, without modifying the underlying vertex data. This transformation approach only requires matrix multiplication operations on the GPU rather than recreating geometry, making it ideal for real-time applications where performance is critical.

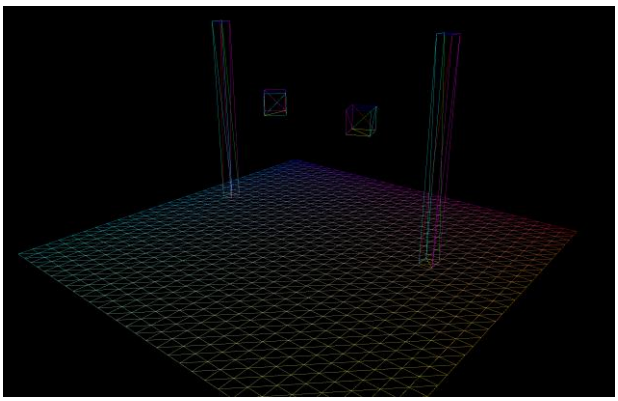
Exercise 2:

Question:

HIERARCHICAL TRANSFORMATIONS

Solution:

```
for (size_t i = 0; i < drawCommands.size(); ++i) {
    const auto& cmd = drawCommands[i];
    ModelPushConstant pc{};

    glm::mat4 modelMatrix;

    if (i == 1 || i == 2) {
        // pillar transformations
        modelMatrix = cmd.modelMatrix;
    }
    else if (i == 3) {
        // First orbiting cube (slower orbit)
        float orbitRadius = 1.2f;
        float orbitSpeed = 0.4f;
        float orbitHeight = 1.0f;

        // Calculate position on orbit circle around the first pillar
        float x = -2.0f + orbitRadius * std::cos(time * orbitSpeed);
        float z = orbitRadius * std::sin(time * orbitSpeed);

        // Create matrix: position + self-rotation + scale
        modelMatrix = glm::translate(glm::mat4(1.0f), glm::vec3(x, orbitHeight, z));
        modelMatrix = modelMatrix * glm::rotate(glm::mat4(1.0f), time, glm::vec3(0.0f, 1.0f, 0.0f));
        modelMatrix = glm::scale(modelMatrix, glm::vec3(0.4f));
    }
```



Reflection:

By iterating over members in the drawCommands struct, transformations can be applied to individual objects within the recordCommandBuffer. The order in which these objects are initialised determines their layer position, with objects created first being in the background and the latter in the foreground.

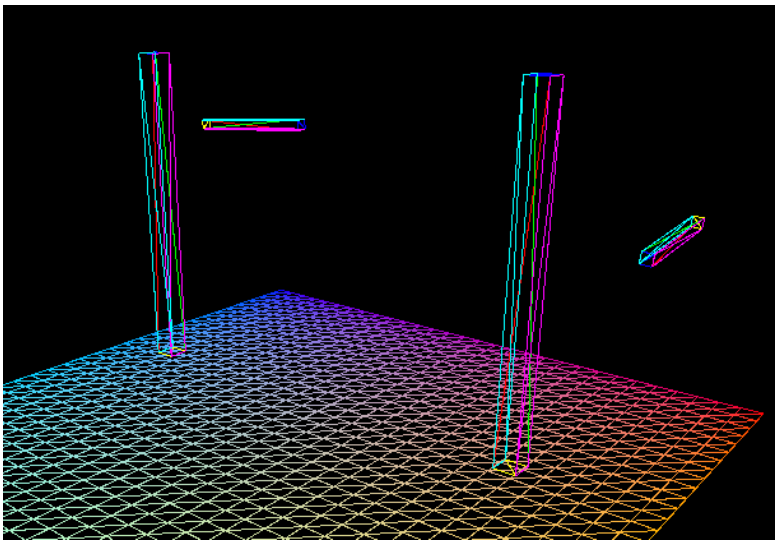
Exercise 3:

Question:

Advanced Rotation - Tangent to Path

Solution:

```
// Second orbiting stick - tangent to circular path (faster)
float orbitRadius = 1.0f;
float orbitSpeed = 1.0f;
float orbitHeight = 1.5f;

// Calculate angle based on time
float angle = time * orbitSpeed;

// Calculate position on orbit circle around the second pillar
float x = 2.0f + orbitRadius * std::cos(angle);
float z = orbitRadius * std::sin(angle);

// Calculate tangent vector
glm::vec3 tangent = glm::normalize(glm::vec3(-std::sin(angle), 0.0f, std::cos(angle)));
glm::vec3 defaultUp = glm::vec3(0.0f, 1.0f, 0.0f);

// Calculate rotation to align with tangent
glm::vec3 rotationAxis = glm::cross(defaultUp, tangent);
float rotationAngle = std::acos(glm::dot(defaultUp, tangent));

// Build transformation matrix
modelMatrix = glm::translate(glm::mat4(1.0f), glm::vec3(x, orbitHeight, z));

if (glm::length(rotationAxis) > 0.001f) {
    modelMatrix = glm::rotate(modelMatrix, rotationAngle, glm::normalize(rotationAxis));
}

// Scale into a stick (thinner and longer than the first one)
modelMatrix = glm::scale(modelMatrix, glm::vec3(0.08f, 1.2f, 0.08f));
```



Reflection:

By calculating the tangent vectors along an orbit, objects can align with its direction of movement. Using the cross products and arc cosine, rotation matrices can be derived to orient the stick tangentially to each point.

Transformations must be applied in order, ensuring translation, rotation and scaling are done correctly.

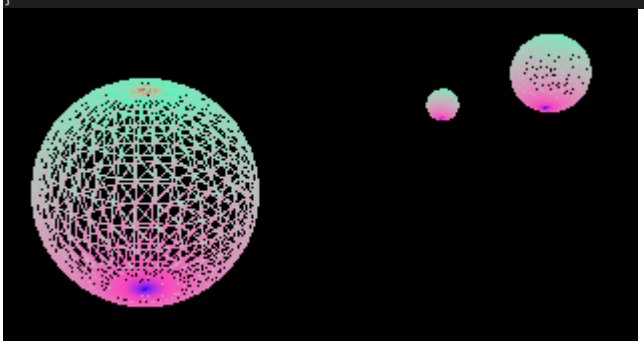
Exercise 4:

Question:

ANIMATING A SOLAR SYSTEM

Solution:

```
else if (i == 1) {
    // Earth - hierarchical transformation relative to Sun
    modelMatrix = glm::mat4(1.0f);

    // 1. Start with Sun's position (origin)
    // 2. Orbital rotation around Sun
    modelMatrix = glm::rotate(modelMatrix, time * earthOrbitSpeed, glm::vec3(0.0f, 1.0f, 0.0f));

    // 3. Translate to orbital distance
    modelMatrix = glm::translate(modelMatrix, glm::vec3(earthOrbitRadius, 0.0f, 0.0f));

    // 4. Axial rotation (Earth's day/night cycle)
    modelMatrix = glm::rotate(modelMatrix, time * earthRotationSpeed, glm::vec3(0.0f, 1.0f, 0.0f));

    // 5. Scale for Earth's size
    modelMatrix = glm::scale(modelMatrix, glm::vec3(0.8f, 0.8f, 0.8f));
}
else if (i == 2) {
    // Moon - hierarchical transformation relative to Earth

    // 1. Start with Earth's transformation
    glm::mat4 earthTransform = glm::mat4(1.0f);
    earthTransform = glm::rotate(earthTransform, time * earthOrbitSpeed, glm::vec3(0.0f, 1.0f, 0.0f));
    earthTransform = glm::translate(earthTransform, glm::vec3(earthOrbitRadius, 0.0f, 0.0f));
    earthTransform = glm::rotate(earthTransform, time * earthRotationSpeed, glm::vec3(0.0f, 1.0f, 0.0f));

    // 2. Apply Moon's orbital rotation relative to Earth
    modelMatrix = earthTransform;
    modelMatrix = glm::rotate(modelMatrix, time * moonOrbitSpeed, glm::vec3(0.0f, 1.0f, 0.0f));

    // 3. Translate to Moon's orbital distance from Earth
    modelMatrix = glm::translate(modelMatrix, glm::vec3(moonOrbitRadius, 0.0f, 0.0f));

    // 4. Scale for Moon's size
    modelMatrix = glm::scale(modelMatrix, glm::vec3(0.3f, 0.3f, 0.3f));
}
```



Reflection:

With the sun at the origin, the earth orbits on the y-axis by (time * earthOrbitSpeed) radians, which is used to calculate the orbital angle. The earth is then translated along the x-axis which, combined with the rotation, creates a circular orbit around the origin.

The moon follows a copy of the earths identity matrix, following the same process to orbit the orbiting object.

Exercise 5:

Question:

UNDERSTANDING VIEW AND PROJECTION

Solution:

```
struct UniformBufferObject {
    alignas(16) glm::vec3 eyePosition;
    alignas(4)  float padding1;
    alignas(16) glm::vec3 lookAtPoint;
    alignas(4)  float padding2;
    alignas(16) glm::vec3 upDirection;
    alignas(4)  float padding3;

    alignas(4)  float fovy;
    alignas(4)  float aspect;
    alignas(4)  float zNear;
    alignas(4)  float zFar;
};
```

```
void HelloTriangleApplication::updateUniformBuffer(uint32_t currentImage) {
    UniformBufferObject ubo{};

    // Camera parameters
    ubo.eyePosition = glm::vec3(8.0f, 6.0f, 8.0f);
    ubo.lookAtPoint = glm::vec3(0.0f, 0.0f, 0.0f);
    ubo.upDirection = glm::vec3(0.0f, 1.0f, 0.0f);

    // Projection parameters
    ubo.fovy = glm::radians(45.0f);
    ubo.aspect = swapChainExtent.width / (float)swapChainExtent.height;
    ubo.zNear = 0.1f;
    ubo.zFar = 100.0f;

    memcpy(uniformBuffersMapped[currentImage], &ubo, sizeof(ubo));
```

Reflection:

Instead of relying on GLM library functions, view and projection matrix construction can be implemented directly in GLSL vertex shaders. This required passing raw camera parameters to the shader and reconstructing the transformation matrices from first principals.

The view matrix construction shows how camera orientation is established through perpendicular axis vectors, where the transformation requires negative dot products for translation, as the world is being transformed into camera space, rather than the camera itself being moved.

Lab 4

Date: 24/10/2025

Exercise 1:

Question:

PREPARING FOR LIGHTING

Solution:

The Vertex.h header was updated to include “normal”.

```
struct Vertex {
    glm::vec3 pos;
    glm::vec3 color;
    glm::vec3 normal;

    static VkVertexInputBindingDescription getBindingDescription() {
        VkVertexInputBindingDescription bindingDescription{};
        bindingDescription.binding = 0;
        bindingDescription.stride = sizeof(Vertex);
        bindingDescription.inputRate = VK_VERTEX_INPUT_RATE_VERTEX;
        return bindingDescription;
    }

    static std::array<VkVertexInputAttributeDescription, 3> getAttributeDescriptions() {
        std::array<VkVertexInputAttributeDescription, 3> attributeDescriptions{};
        attributeDescriptions[0] = { 0, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, pos) };
        attributeDescriptions[1] = { 1, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, color) };
        attributeDescriptions[2] = { 2, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, normal) };
        return attributeDescriptions;
```

The CreateCube function was updated to generate 36 unique vertices, without using indices. Achieved by making 6 faces made up of 2 triangles which all share the same normal.

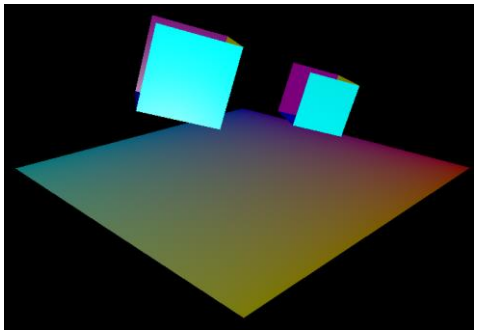
```
GeometryGenerator::MeshData GeometryGenerator::CreateCube(
    float width, float height, float depth)
{
    MeshData meshData;

    float w = width * 0.5f;
    float h = height * 0.5f;
    float d = depth * 0.5f;

    // Reserve space for 36 vertices (6 faces * 6 vertices per face)
    meshData.vertices.reserve(36);
    // No indices needed for non-indexed rendering
    meshData.indices.clear();

    // Front face (facing +Z, normal = {0, 0, 1})
    glm::vec3 frontNormal = glm::vec3(0.0f, 0.0f, 1.0f);
    glm::vec3 frontColor = glm::vec3(1.0f, 0.0f, 0.0f); // Red

    // Triangle 1
    meshData.vertices.push_back(Vertex{ {-w, -h,  d}, frontColor, frontNormal });
    meshData.vertices.push_back(Vertex{ { w, -h,  d}, frontColor, frontNormal });
    meshData.vertices.push_back(Vertex{ { w,  h,  d}, frontColor, frontNormal });

    // Triangle 2
    meshData.vertices.push_back(Vertex{ {-w, -h,  d}, frontColor, frontNormal });
    meshData.vertices.push_back(Vertex{ { w,  h,  d}, frontColor, frontNormal });
    meshData.vertices.push_back(Vertex{ {-w,  h,  d}, frontColor, frontNormal });
```

The vertex shader was updated to include new data for lighting.

```
layout(binding = 0) uniform CameraParameters {
    vec3 eyePosition;       // Camera position
    float padding1;
    vec3 lookAtPoint;       // Point camera is looking at
    float padding2;
    vec3 upDirection;       // Up vector
    float padding3;

    // Projection parameters
    float fovy;             // Field of view (radians)
    float aspect;           // Aspect ratio (width/height)
    float zNear;            // Near clipping plane
    float zFar;             // Far clipping plane

    // Lighting parameters
    vec3 lightPos;
    vec3 eyePos;
```

This exercise demonstrated the implementation of per-vertex lighting using the Phong illumination model by extending the vertex shader to pass normals, positions, and lighting parameters to the fragment shader for accurate light calculations.

The key architectural decision was structuring the UniformBufferObject with proper memory alignment (using alignas(16)) to include both camera parameters and lighting data (lightPos and eyePos), ensuring efficient GPU memory access patterns. By updating Vertex.h to include a third attribute description for normals and modifying the fragment shader to compute ambient, diffuse, and specular components, the solar system scene gained realistic lighting that responds to surface orientation relative to the light source.



Exercise 2:

Question:

PER-VERTEX DIFFUSE LIGHTING

Solution:

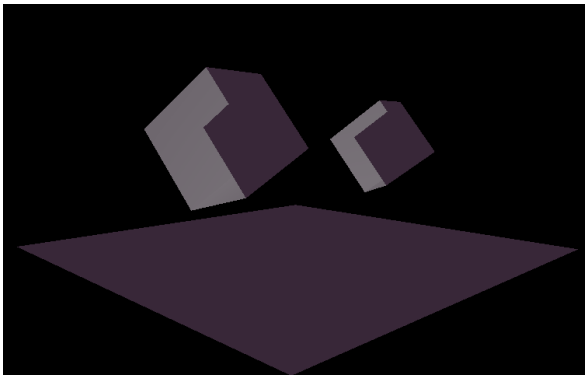
This implementation added a basic Phong reflection model to the vertex shader, calculating ambient and diffuse lighting components for each vertex before passing the computed colour to the fragment shader.

```
vec3 worldNormal = normalize(normalMatrix * inNormal);

// Define light and material properties
vec3 lightColor = vec3(1.0, 1.0, 1.0);      // White light
vec3 ambientMaterial = vec3(0.2, 0.1, 0.2); // Purple-tinted ambient
vec3 diffMaterial = vec3(1.0, 1.0, 1.0);    // White diffuse material

// Ambient component
vec3 ambient = ambientMaterial * lightColor;

// Diffuse component
vec3 norm = normalize(worldNormal);
vec3 lightDir = normalize(ubo.lightPos - worldPos.xyz);
float diff = max(dot(norm, lightDir), 0.0);
vec3 diffuse = diff * diffMaterial * lightColor;

// Combine ambient and diffuse lighting
fragColor = ambient + diffuse;
```

The vertex shader transforms both the vertex position and normal vector to world space using the model matrix, then defines material properties including an ambient component of vec3(0.2, 0.1, 0.2) and a diffuse component of vec3(1.0, 1.0, 1.0), with white light at vec3(1.0, 1.0, 1.0).

The diffuse lighting calculation uses the Lambertian reflectance model, computing the dot product between the normalized surface normal and the light direction vector (from the surface point to the light position), clamped to prevent negative values. The final vertex colour combines the ambient term (ambient material multiplied by light colour) with the diffuse term (diffuse factor multiplied by diffuse material and light colour), and this computed colour is linearly interpolated across triangle faces during rasterization.

The fragment shader was simplified to a pass-through implementation that receives the interpolated fragColour from the vertex shader and outputs it directly as the final pixel colour with full opacity.



Exercise 3:

Question:

PER-FRAGMENT DIFFUSE LIGHTING

Solution:

This implementation relocated the ambient and diffuse lighting calculations from the vertex shader to the fragment shader, transitioning from Gouraud shading to Phong shading for improved visual quality.

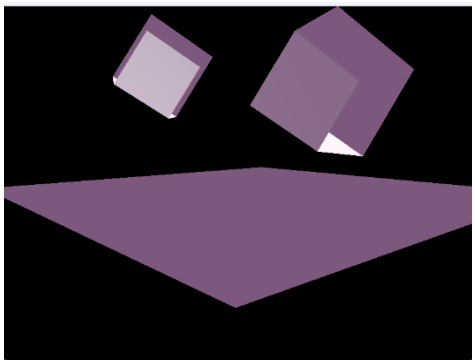
```
void main() {
    // Build view matrix from camera parameters
    mat4 view = buildViewMatrix(ubo.eyePosition, ubo.lookAtPoint, ubo.upDirection);

    // Build perspective projection matrix
    mat4 proj = buildPerspectiveMatrix(ubo.fovy, ubo.aspect, ubo.zNear, ubo.zFar);

    // Transform position to world space
    vec4 worldPos = pushConstants.model * vec4(inPosition, 1.0);

    // Apply transformations: Projection * View * Model * Position
    gl_Position = proj * view * worldPos;

    // Pass world position to fragment shader
    fragWorldPos = worldPos.xyz;

    // Transform normal to world space and pass to fragment shader
    mat3 normalMatrix = transpose(inverse(mat3(pushConstants.model)));
    fragWorldNormal = normalMatrix * inNormal;

    // Pass vertex color to fragment shader
    fragColor = inColor;
```

```
uboLayoutBinding.stageFlags = VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT;
```

The vertex shader was modified to pass three interpolated outputs to the fragment shader: the vertex color (fragColor), world-space position (fragWorldPos), and world-space normal (fragWorldNormal), with the normal transformed using the normal matrix transpose(inverse(mat3(model))).

The fragment shader was updated to include the CameraParameters uniform buffer block to access the light position, and it now performs per-pixel lighting calculations using the interpolated world position and normal vectors.

```
void main() {
    // Define light and material properties
    vec3 lightColor = vec3(1.0, 1.0, 1.0);      // White light
    vec3 ambientMaterial = vec3(0.2, 0.1, 0.2); // Purple-tinted ambient
    vec3 diffMaterial = vec3(1.0, 1.0, 1.0);    // White diffuse material

    // Ambient component
    vec3 ambient = ambientMaterial * lightColor;

    // Diffuse component (per-fragment calculation)
    vec3 norm = normalize(fragWorldNormal);
    vec3 lightDir = normalize(ubo.lightPos - fragWorldPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = diff * diffMaterial * lightColor;

    // Combine ambient and diffuse lighting
    vec3 finalColor = ambient + diffuse;

    outColor = vec4(finalColor, 1.0);
}
```



The lighting model computes an ambient term with material vec3(0.2, 0.1, 0.2) and a diffuse term using the Lambertian reflectance formula max(dot(norm, lightDir), 0.0), combining both components to produce the final pixel colour.

This approach produces significantly smoother lighting gradients across surfaces compared to per-vertex lighting, as the lighting calculation is now performed for every fragment rather than being linearly interpolated from vertex values.

Exercise 4:

Question:

ADDING PER-VERTEX SPECULAR LIGHTING

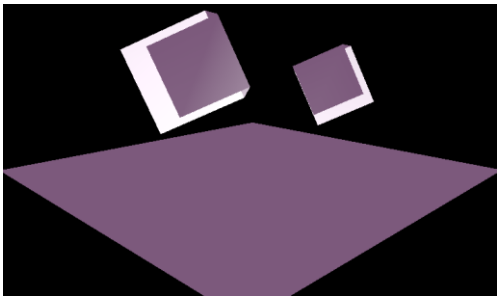
Solution:

Exercise 4 implements the complete Phong lighting model (ambient + diffuse + specular) entirely in the vertex shader, with all three components calculated per-vertex and then interpolated across triangle surfaces.

```
// ===== LIGHTING CALCULATIONS (PER-VERTEX) =====

// Define light and material properties
vec3 lightColor = vec3(1.0, 1.0, 1.0);      // White light
vec3 ambientMaterial = vec3(0.2, 0.1, 0.2); // Purple-tinted ambient
vec3 diffMaterial = vec3(1.0, 1.0, 1.0);    // White diffuse material
vec3 specMaterial = vec3(1.0);              // White specular material

// Ambient component
vec3 ambient = ambientMaterial * lightColor;

// Diffuse component
vec3 lightDir = normalize(ubo.lightPos - worldPos.xyz);
float diff = max(dot(norm, lightDir), 0.0);
vec3 diffuse = diff * diffMaterial * lightColor;

// Specular calculation
vec3 viewDir = normalize(ubo.eyePos - worldPos.xyz);
vec3 reflectDir = normalize(reflect(-lightDir, norm));
float shininess = 32.0;
float spec = pow(max(dot(reflectDir, viewDir), 0.0), shininess);
vec3 specular = specMaterial * lightColor * spec;

// Combine all components (ambient + diffuse + specular)
fragColor = ambient + diffuse + specular;
```

The vertex shader calculates the view direction from each vertex to the camera, computes the reflection direction using GLSL's reflect() function, and applies the Phong specular formula with a shininess value of 32.0 to create bright highlights where the reflection aligns with the viewer's eye.

The fragment shader is simplified to a pass-through, simply outputting the interpolated colour without performing any lighting calculations, while the C++ code ensures the uniform buffer is accessible to both shader stages via the descriptor set layout.



The result is shiny specular highlights on the rotating cubes, though the highlights appear less smooth than per-fragment calculations since they're computed only at vertices and interpolated.

Exercise 5:

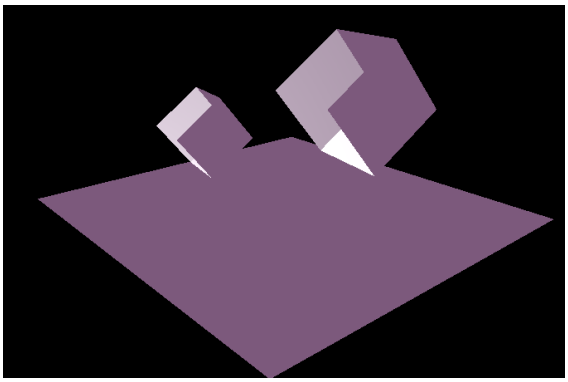
Question:

Exercise 5 completes the Phong reflection model by adding specular highlights calculated per-fragment in the fragment shader, building directly upon Exercise 3's per-fragment diffuse lighting implementation.

```
void main() {
    // Define light and material properties
    vec3 lightColor = vec3(1.0, 1.0, 1.0);      // White light
    vec3 ambientMaterial = vec3(0.2, 0.1, 0.2); // Purple-tinted ambient
    vec3 diffMaterial = vec3(1.0, 1.0, 1.0);    // White diffuse material
    vec3 specMaterial = vec3(1.0);              // White specular material

    // Ambient component
    vec3 ambient = ambientMaterial * lightColor;

    // Diffuse component
    vec3 norm = normalize(fragWorldNormal);
    vec3 lightDir = normalize(ubo.lightPos - fragWorldPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = diff * diffMaterial * lightColor;

    // Specular component
    vec3 viewDir = normalize(ubo.eyePos - fragWorldPos);
    vec3 reflectDir = normalize(reflect(-lightDir, norm));
    float shininess = 32.0;
    float spec = pow(max(dot(reflectDir, viewDir), 0.0), shininess);
    vec3 specular = specMaterial * lightColor * spec;

    // Combine all three components (ambient + diffuse + specular)
    vec3 finalColor = ambient + diffuse + specular;

    outColor = vec4(finalColor, 1.0);
```

The fragment shader now computes the view direction from each fragment to the camera (ubo.eyePos), calculates the reflection direction using GLSL's reflect() function, and applies the Phong specular formula with a shininess value of 32.0 to produce bright, sharp highlights where the reflection angle aligns with the view direction.

Unlike Exercise 4's per-vertex specular approach, this per-pixel calculation results in much smoother and more accurate highlights, especially visible on surfaces where specular reflections change rapidly across polygons.



The final output combines all three Phong components—ambient, diffuse, and specular—calculated entirely per-fragment, producing realistic shiny surfaces with crisp, high-quality specular highlights that move dynamically as the cubes rotate and the camera moves.

Exercise 6:

Question:

MULTIPLE LIGHTS AND MATERIALS

Draw three cube objects with three different surface material properties. Illuminate these cubes with two light sources: one is a static white light; the other is a red light rotating dynamically by the y-axis.

Solution:

The Ubo now includes position and colour parameters for the red and white light and a PushConstants struct is created to send per-draw material.

```
struct UniformBufferObject {
    // White light
    alignas(16) glm::vec3 lightPos;
    alignas(16) glm::vec3 lightColor;

    // Red light
    alignas(16) glm::vec3 redLightPos;
    alignas(16) glm::vec3 redLightColor;

    // Camera / view
    alignas(16) glm::vec3 eyePosition;
    alignas(4)  float padding1;
    alignas(16) glm::vec3 lookAtPoint;
    alignas(4)  float padding2;
    alignas(16) glm::vec3 upDirection;
    alignas(4)  float padding3;

    // Projection params
    alignas(4)  float fovy;
    alignas(4)  float aspect;
    alignas(4)  float zNear;
    alignas(4)  float zFar;
};

struct PushConstants {
    alignas(16) glm::mat4 model;

    // material (sent to fragment shader)
    alignas(16) glm::vec3 ambient;
    alignas(16) glm::vec3 diffuse;
    alignas(16) glm::vec3 specular;
    alignas(4)  float shininess;
};
```

```
void HelloTriangleApplication::updateUniformBuffer(uint32_t currentImage) {
    UniformBufferObject ubo{};

    // Camera parameters using camera control variables
    ubo.eyePosition = cameraPos;
    ubo.lookAtPoint = cameraPos + cameraFront;
    ubo.upDirection = cameraUp;

    // Projection parameters
    ubo.fovy = glm::radians(45.0f);
    ubo.aspect = swapChainExtent.width / (float)swapChainExtent.height;
    ubo.zNear = 0.1f;
    ubo.zFar = 100.0f;

    // Static white light (position and color)
    ubo.lightPos = glm::vec3(5.0f, -1.0f, 5.0f);
    //ubo.lightPos = glm::vec3(5.0f, 5.0f, -5.0f);
    ubo.lightColor = glm::vec3(1.0f, 1.0f, 1.0f);

    // Rotating red light: compute angle from elapsed time and place it on a circle around Y
    auto currentTime = std::chrono::high_resolution_clock::now();
    float time = std::chrono::duration<float, std::chrono::seconds::period>(currentTime - startTime).count();

    const float radius = 5.0f;                      // radius of rotation
    const float height = 3.0f;                      // y position of red light
    const float angularSpeed = glm::radians(45.0f);    // 45 degrees per second (radians/sec)
    float angle = time * angularSpeed;

    ubo.redLightPos = glm::vec3(std::cos(angle) * radius, height, std::sin(angle) * radius);
    ubo.redLightColor = glm::vec3(1.0f, 0.0f, 0.0f);

    memcpy(uniformBuffersMapped[currentImage], &ubo, sizeof(ubo));
}
```

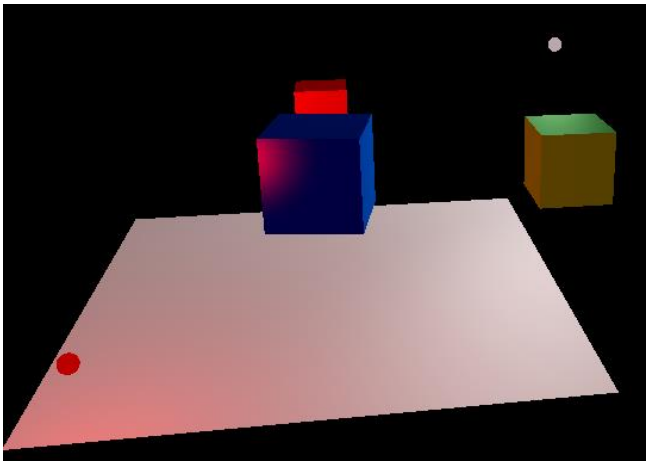
The ubo update now calculated the position of the red light's orbit.

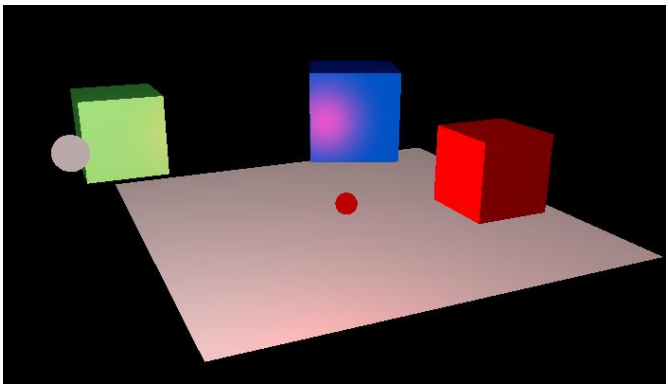
The fragment shade produces Phong lighting and accumulates contribution from white and red light, using material values from push constants for ambient, diffuse, specular and shininess.

```
void main() {
    vec3 N = normalize(vNormal);
    vec3 V = normalize(ubo.eyePosition - vFragPos);

    vec3 color = vec3(0.0);

    // White static light
    {
        vec3 L = normalize(ubo.lightPos - vFragPos);
        float diff = max(dot(N, L), 0.0);
        vec3 R = reflect(-L, N);
        float spec = 0.0;
        if (diff > 0.0) {
            spec = pow(max(dot(R, V), 0.0), pc.shininess);
        }
        vec3 ambient = pc.ambient * ubo.lightColor;
        vec3 diffuse = pc.diffuse * diff * ubo.lightColor;
        vec3 specular = pc.specular * spec * ubo.lightColor;
        color += ambient + diffuse + specular;
    }

    // Red secondary light
    {
        vec3 L = normalize(ubo.redLightPos - vFragPos);
        float diff = max(dot(N, L), 0.0);
        vec3 R = reflect(-L, N);
        float spec = 0.0;
        if (diff > 0.0) {
            spec = pow(max(dot(R, V), 0.0), pc.shininess);
        }
        vec3 ambient = pc.ambient * ubo.redLightColor * 0.25; // reduce ambient contribution of red
        vec3 diffuse = pc.diffuse * diff * ubo.redLightColor;
        vec3 specular = pc.specular * spec * ubo.redLightColor;
        color += ambient + diffuse + specular;
    }

    // tone clamp
    color = clamp(color, 0.0, 1.0);
    outColor = vec4(color, 1.0);
```

The vertex shader computes world-space vFragPos and transformed normal, builds view/proj from Ubo.

The DrawCommand is updated to include a materialID, which can be used to assign different push constants.





```
void main() {
    mat4 model = pc.model;
    vec4 worldPos = model * vec4(inPosition, 1.0);
    vFragPos = worldPos.xyz;

    // normal transform (account for non-uniform scale)
    vNormal = normalize(mat3(transpose(inverse(model))) * inNormal);

    // build view & projection from UBO camera parameters
    mat4 view = lookAt(ubo.eyePosition, ubo.lookAtPoint, ubo.upDirection);
    mat4 proj = perspective(ubo.fovy, ubo.aspect, ubo.zNear, ubo.zFar);

    // Vulkan clip space has inverted Y
    // proj[1][1] *= -1.0;

    gl_Position = proj * view * worldPos;
}
```

```
// Default material (used for non-cube objects like the grid)
pc.ambient = glm::vec3(0.05f);
pc.diffuse = glm::vec3(0.6f);
pc.specular = glm::vec3(0.3f);
pc.shininess = 8.0f;

// Choose material based on materialId (preserved from earlier)
switch (cmd.materialId) {
case 1:
    // Cube A: red plastic-like
    pc.ambient = glm::vec3(0.1f, 0.0f, 0.0f);
    pc.diffuse = glm::vec3(0.6f, 0.0f, 0.0f);
    pc.specular = glm::vec3(0.6f, 0.6f, 0.6f);
    pc.shininess = 32.0f;
    break;
case 2:
    // Cube B: green glossy
    pc.ambient = glm::vec3(0.0f, 0.05f, 0.0f);
    pc.diffuse = glm::vec3(0.2f, 0.7f, 0.2f);
    pc.specular = glm::vec3(0.3f, 0.8f, 0.3f);
    pc.shininess = 64.0f;
    break;
case 3:
    // Cube C: metallic blue
    pc.ambient = glm::vec3(0.0f, 0.0f, 0.05f);
    pc.diffuse = glm::vec3(0.0f, 0.1f, 0.6f);
    pc.specular = glm::vec3(0.8f, 0.8f, 0.9f);
    pc.shininess = 128.0f;
    break;
default:
    break;
}
```

Lab 5

Date: 31/10/25

Exercise 1:

Question:

PREPARING THE APPLICATION FOR TEXTURES

Solution:

The stb_image header is included in the project.

```
#define STB_IMAGE_IMPLEMENTATION
#include <stb_image.h>
```

The vector struct and input description is updated to include texCoord.

```
struct Vertex {
    glm::vec3 pos;
    glm::vec3 color;
    glm::vec3 normal;

    glm::vec2 texCoord;

    static VkVertexInputBindingDescription getBindingDescription() {
        VkVertexInputBindingDescription bindingDescription{};
        bindingDescription.binding = 0;
        bindingDescription.stride = sizeof(Vertex);
        bindingDescription.inputRate = VK_VERTEX_INPUT_RATE_VERTEX;
        return bindingDescription;
    }

    static std::array<VkVertexInputAttributeDescription, 4> getAttributeDescriptions() {
        std::array<VkVertexInputAttributeDescription, 4> attributeDescriptions{};
        attributeDescriptions[0] = { 0, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, pos) };
        attributeDescriptions[1] = { 1, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, color) };
        attributeDescriptions[2] = { 2, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, normal) };
        attributeDescriptions[3] = { 3, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, texCoord) };

        return attributeDescriptions;
```

UV coordinates are then included in the cube data.

```
meshData.vertices.push_back(Vertex{ {-w, -h,  d}, frontColor, frontNormal, {0.0f, 1.0f} }); // BL
meshData.vertices.push_back(Vertex{ { w,  h,  d}, frontColor, frontNormal, {1.0f, 0.0f} }); // TR
meshData.vertices.push_back(Vertex{ { w, -h,  d}, frontColor, frontNormal, {1.0f, 1.0f} }); // BR

meshData.vertices.push_back(Vertex{ {-w, -h,  d}, frontColor, frontNormal, {0.0f, 1.0f} }); // BL
meshData.vertices.push_back(Vertex{ {-w,  h,  d}, frontColor, frontNormal, {0.0f, 0.0f} }); // TL
meshData.vertices.push_back(Vertex{ { w,  h,  d}, frontColor, frontNormal, {1.0f, 0.0f} }); // TR
```

Exercise 2:

Question:

LOADING AND CREATING VULKAN IMAGE RESOURCES

Solution:

The Vulkan image and pixel-storage members are added to the HelloTriangleApplication class, along with the staging buffer.

```
// --- Texture resources (added) ---
VkImage textureImage = VK_NULL_HANDLE;
VkDeviceMemory textureImageMemory = VK_NULL_HANDLE;
VkImageView textureImageView = VK_NULL_HANDLE;
VkSampler textureSampler = VK_NULL_HANDLE;

// Temporary CPU-side storage for loaded image (freed after upload)
stbi_uc* texturePixels = nullptr;
int texWidth = 0;
int texHeight = 0;
int texChannels = 0;
VkDeviceSize textureImageSize = 0;
```

```
// Staging buffer for texture upload (persist until image upload completes)
VkBuffer textureStagingBuffer = VK_NULL_HANDLE;
VkDeviceMemory textureStagingBufferMemory = VK_NULL_HANDLE;
```

The image is then loaded with stb_image in initVulkan(), after createCommandPool(). Free CPU pixel memory and destroy Vulkan objects during cleanup.

```
VkShaderModule createShaderModule(const std::vector<char>& code);
void createBuffer(VkDeviceSize size, VkBufferUsageFlags usage, VkMemoryPropertyFlags properties, VkBuffer& buffer,
  VkDeviceMemory& bufferMemory);
void createImage(uint32_t width, uint32_t height, VkFormat format, VkImageTiling tiling, VkImageUsageFlags usage,
  VkMemoryPropertyFlags properties, VkImage& image, VkDeviceMemory& imageMemory);
VkImageView createImageView(VkImage image, VkFormat format, VkImageAspectFlags aspectFlags);
void createTextureImageView();
void createTextureSampler();
void transitionImageLayout(VkImage image, VkFormat format, VkImageLayout oldLayout, VkImageLayout newLayout);
void copyBufferToImage(VkBuffer buffer, VkImage image, uint32_t width, uint32_t height);
void copyBuffer(VkBuffer srcBuffer, VkBuffer dstBuffer, VkDeviceSize size);
uint32_t findMemoryType(uint32_t typeFilter, VkMemoryPropertyFlags properties);
```

The staging buffer is created within initVulkan(), where the pixels are copied into it.

```
// Create staging buffer (HOST_VISIBLE | HOST_COHERENT) and copy pixels into it
createBuffer(
    textureImageSize,
    VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
    VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
    textureStagingBuffer,
    textureStagingBufferMemory
);

void* mapped = nullptr;
vkMapMemory(device, textureStagingBufferMemory, 0, textureImageSize, 0, &mapped);
std::memcpy(mapped, texturePixels, static_cast<size_t>(textureImageSize));
vkUnmapMemory(device, textureStagingBufferMemory);

// Create GPU image and allocate device-local memory
createImage(
    static_cast<uint32_t>(texWidth),
    static_cast<uint32_t>(texHeight),
    VK_FORMAT_R8G8B8A8_SRGB,
    VK_IMAGE_TILING_OPTIMAL,
    VK_IMAGE_USAGE_TRANSFER_DST_BIT | VK_IMAGE_USAGE_SAMPLED_BIT,
    VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT,
    textureImage,
    textureImageMemory
);

// Create image view for shader access
createTextureImageView();

// Transition image layout and copy from staging buffer
transitionImageLayout(textureImage, VK_FORMAT_R8G8B8A8_SRGB, VK_IMAGE_LAYOUT_UNDEFINED,
  VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL);

// Copy staging buffer to GPU image
copyBufferToImage(textureStagingBuffer, textureImage, static_cast<uint32_t>(texWidth), static_cast<uint32_t>(texHeight));

// Transition to shader-read layout
transitionImageLayout(textureImage, VK_FORMAT_R8G8B8A8_SRGB, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
  VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL);

createTextureSampler();

// Clean up staging buffer now that texture is uploaded to GPU
vkDestroyBuffer(device, textureStagingBuffer, nullptr);
vkFreeMemory(device, textureStagingBufferMemory, nullptr);
textureStagingBuffer = VK_NULL_HANDLE;
textureStagingBufferMemory = VK_NULL_HANDLE;

// Free CPU-side pixels now that the data is in the staging buffer
stbi_image_free(texturePixels);
texturePixels = nullptr;
```

Reflection:

I implemented a complete texture loading and GPU upload pipeline for Vulkan. First, I used the stb_image library to load a texture file from disk, then created a host-visible staging buffer and copied the raw pixel data into it for intermediate storage. Next, I created a device-local VkImage with optimal tiling on the GPU, transitioned its layout from undefined to transfer-destination-optimal using pipeline barriers, and issued a vkCmdCopyBufferToImage command to transfer the data from the staging buffer to the GPU image. After the transfer completed, I transitioned the image layout again to shader-read-only-optimal so fragment shaders can sample it. Finally, I created a VkImageView to provide a view into the texture data and a VkSampler configured with linear filtering, anisotropic filtering enabled, and repeat addressing mode—making the texture ready to be bound to descriptor sets and accessed in shaders for rendering.

Exercise 3:

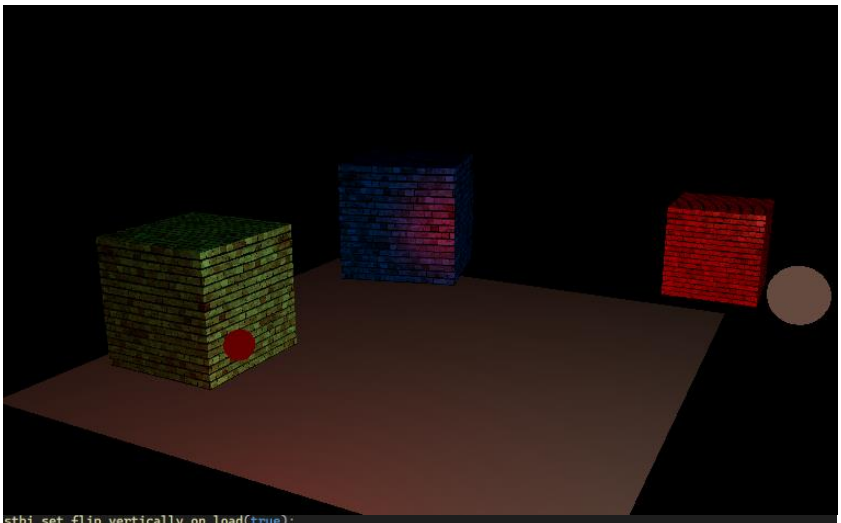
Question:

BINDING AND SHADER UPDATES

Solution:

The descriptor set layout was expanded from single to dual-binding configuration by adding a combined image sampler at binding 1 for fragment shader texture access, while modifying the existing UBO binding to be accessible from both vertex and fragment stages. The descriptor pool was updated to allocate resources for both uniform buffers and combined image samplers across all frames in flight. The descriptor sets were modified to write both the UBO and texture sampler simultaneously using dual write operations per frame.

The vertex shader was updated to receive texture coordinates at location 3 from the vertex buffer, pass them through to the fragment shader via interpolation, and the helper functions were repositioned before main() to satisfy GLSL compilation requirements. The fragment shader was updated to declare the texture sampler at binding 1, receive interpolated texture coordinates, sample the texture, and modulate the Phong lighting result with the texture colour to achieve textured surfaces with preserved lighting response.



```
stbi_set_flip_vertically_on_load(true);

int w = 0, h = 0, channels = 0;
stbi_uc* pixels = stbi_load("coin.jpg", &w, &h, &channels, STBI_rgb_alpha);
if (!pixels) {
    throw std::runtime_error("failed to load texture image!");
}

texturePixels = pixels;
texWidth = w;
texHeight = h;
texChannels = channels;
textureImageSize = static_cast<VkDeviceSize>(w) * static_cast<VkDeviceSize>(h) * 4; // 4 bytes per pixel (RGBA)

// Create staging buffer (HOST_VISIBLE | HOST_COHERENT) and copy pixels into it
createBuffer(
    textureImageSize,
    VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
    VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
    textureStagingBuffer,
    textureStagingBufferMemory
);

void* mapped = nullptr;
vkMapMemory(device, textureStagingBufferMemory, 0, textureImageSize, 0, &mapped);
std::memcpy(mapped, texturePixels, static_cast<size_t>(textureImageSize));
vkUnmapMemory(device, textureStagingBufferMemory);

// Create GPU image and allocate device-local memory
createImage(
    static_cast<uint32_t>(texWidth),
    static_cast<uint32_t>(texHeight),
    VK_FORMAT_R8G8B8A8_SRGB,
    VK_IMAGE_TILING_OPTIMAL,
    VK_IMAGE_USAGE_TRANSFER_DST_BIT | VK_IMAGE_USAGE_SAMPLED_BIT,
    VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT,
    textureImage,
    textureImageMemory
);
```

Exercise 4:

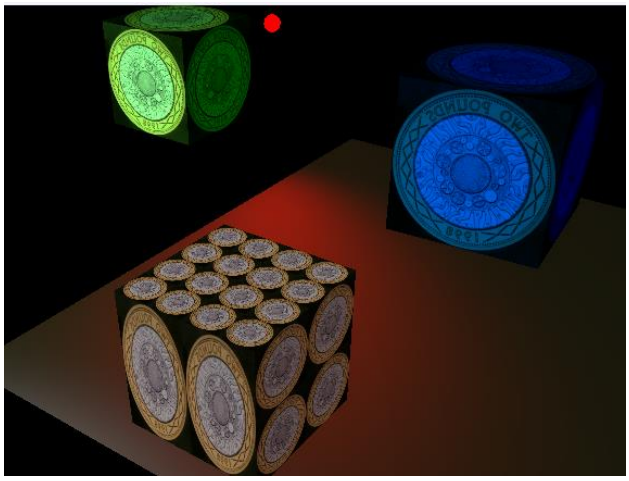
Question:

TEXTURE WRAPPING MODE

Solution:

Each face of the multi-textured cube gets per-vertex UVs that span ranges greater than [0,1] (set in CreateMultiTexCube()), so the interpolated per-pixel texture coordinates cover N×M tiles across a face.

```
// Front face (facing +Z, normal = {0, 0, 1}) - 1 coin pattern
glm::vec3 frontNormal = glm::vec3(0.0f, 0.0f, 1.0f);
glm::vec3 frontColor = glm::vec3(1.0f, 0.0f, 0.0f); // Red

meshData.vertices.push_back(Vertex{ {-w, -h,  d}, frontColor, frontNormal, {0.0f, 1.0f} }); // BL
meshData.vertices.push_back(Vertex{ { w,  h,  d}, frontColor, frontNormal, {1.0f, 0.0f} }); // TR
meshData.vertices.push_back(Vertex{ { w, -h,  d}, frontColor, frontNormal, {1.0f, 1.0f} }); // BR

meshData.vertices.push_back(Vertex{ {-w, -h,  d}, frontColor, frontNormal, {0.0f, 1.0f} }); // BL
meshData.vertices.push_back(Vertex{ {-w,  h,  d}, frontColor, frontNormal, {0.0f, 0.0f} }); // TL
meshData.vertices.push_back(Vertex{ { w,  h,  d}, frontColor, frontNormal, {1.0f, 0.0f} }); // TR

// Back face (facing -Z, normal = {0, 0, -1}) - 2x2 coin pattern
glm::vec3 backNormal = glm::vec3(0.0f, 0.0f, -1.0f);
glm::vec3 backColor = glm::vec3(0.0f, 1.0f, 0.0f); // Green

meshData.vertices.push_back(Vertex{ { w, -h, -d}, backColor, backNormal, {0.0f, 2.0f} }); // BL
meshData.vertices.push_back(Vertex{ {-w,  h, -d}, backColor, backNormal, {2.0f, 0.0f} }); // TR
meshData.vertices.push_back(Vertex{ {-w, -h, -d}, backColor, backNormal, {2.0f, 2.0f} }); // BR

meshData.vertices.push_back(Vertex{ { w, -h, -d}, backColor, backNormal, {0.0f, 2.0f} }); // BL
meshData.vertices.push_back(Vertex{ { w,  h, -d}, backColor, backNormal, {0.0f, 0.0f} }); // TL
meshData.vertices.push_back(Vertex{ {-w,  h, -d}, backColor, backNormal, {2.0f, 0.0f} }); // TR
```



The sampler is created with VK_SAMPLER_ADDRESS_MODE_REPEAT, so UV components above 1.0 wrap and the fragment shader sampling texture(sampler, inTexCoord) produces repeated coin tiles.

The coin image is uploaded with stb_image as VK_FORMAT_R8G8B8A8_SRGB and bound as a combined image sampler, while the cube's material uses a white diffuse (no tint), letting the sampled coin colours appear unchanged.

Exercise 5:

Question:

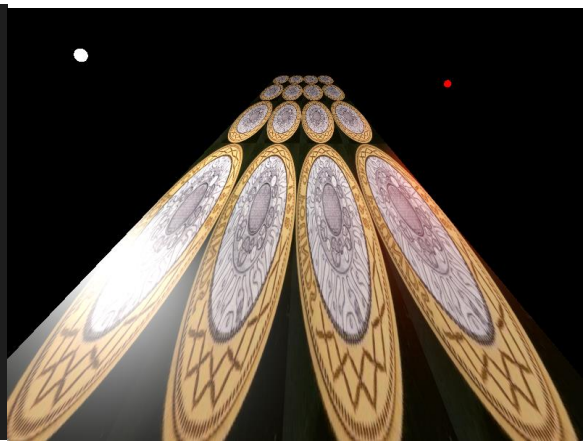
Texture filtering techniques

Solution:

Anisotropic Filtering (single mip level)

This technique enables anisotropyEnable = VK_TRUE with maxAnisotropy set to the device maximum (typically 16), taking multiple anisotropic samples along the direction of maximum texture compression. Your road geometry, which extends along the ground plane at an oblique angle to the camera, will appear significantly sharper and more detailed in the distance compared to standard bilinear/trilinear filtering, as anisotropic filtering specifically addresses the blurring that occurs when textures are viewed at grazing angles.

```
void HelloTriangleApplication::createTextureSampler() {
    VkPhysicalDeviceProperties properties{};
    vkGetPhysicalDeviceProperties(physicalDevice, &properties);

    VkSamplerCreateInfo samplerInfo{};
    samplerInfo.sType = VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO;
    samplerInfo.pNext = nullptr;
    samplerInfo.flags = 0;

    samplerInfo.magFilter = VK_FILTER_LINEAR;
    samplerInfo.minFilter = VK_FILTER_LINEAR;

    samplerInfo.addressModeU = VK_SAMPLER_ADDRESS_MODE_REPEAT;
    samplerInfo.addressModeV = VK_SAMPLER_ADDRESS_MODE_REPEAT;
    samplerInfo.addressModeW = VK_SAMPLER_ADDRESS_MODE_REPEAT;

    samplerInfo.anisotropyEnable = VK_TRUE;
    samplerInfo.maxAnisotropy = properties.limits.maxSamplerAnisotropy;

    samplerInfo.borderColor = VK_BORDER_COLOR_INT_OPAQUE_BLACK;
    samplerInfo.unnormalizedCoordinates = VK_FALSE;
    samplerInfo.compareEnable = VK_FALSE;
    samplerInfo.compareOp = VK_COMPARE_OP_ALWAYS;

    samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_LINEAR;
    samplerInfo.mipLodBias = 0.0f;
    samplerInfo.minLod = 0.0f;
    samplerInfo.maxLod = 0.0f;
```



Nearest Neighbour

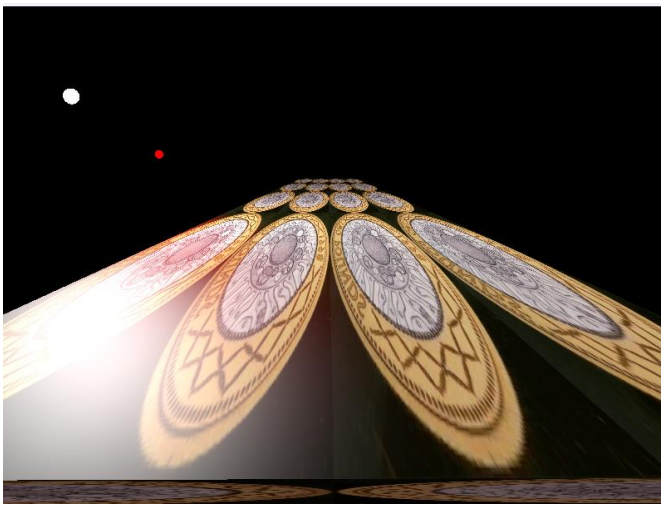
This approach uses VK_FILTER_NEAREST for both magnification and minification, selecting the single closest texel without any interpolation. The result is a sharp, blocky appearance where individual pixels are clearly visible, making it ideal for pixel-art aesthetics but producing visible aliasing artifacts on your textured road, especially at oblique viewing angles.

```
void HelloTriangleApplication::createTextureSampler() {
    VkPhysicalDeviceProperties properties{};
    vkGetPhysicalDeviceProperties(physicalDevice, &properties);

    VkSamplerCreateInfo samplerInfo{};
    samplerInfo.sType = VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO;
    samplerInfo.pNext = nullptr;
    samplerInfo.flags = 0;

    // Nearest-neighbour filtering for both magnification and minification
    samplerInfo.magFilter = VK_FILTER_NEAREST;
    samplerInfo.minFilter = VK_FILTER_NEAREST;

    samplerInfo.addressModeU = VK_SAMPLER_ADDRESS_MODE_REPEAT;
    samplerInfo.addressModeV = VK_SAMPLER_ADDRESS_MODE_REPEAT;
    samplerInfo.addressModeW = VK_SAMPLER_ADDRESS_MODE_REPEAT;

    // Disable anisotropic filtering for pure nearest-neighbour
    samplerInfo.anisotropyEnable = VK_FALSE;
    samplerInfo.maxAnisotropy = 1.0f;

    samplerInfo.borderColor = VK_BORDER_COLOR_INT_OPAQUE_BLACK;
    samplerInfo.unnormalizedCoordinates = VK_FALSE;
    samplerInfo.compareEnable = VK_FALSE;
    samplerInfo.compareOp = VK_COMPARE_OP_ALWAYS;

    // Use nearest mipmap selection
    samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_NEAREST;
    samplerInfo.mipLodBias = 0.0f;
    samplerInfo.minLod = 0.0f;
    samplerInfo.maxLod = 0.0f;
```

Biliniar filtering

This configuration uses VK_FILTER_LINEAR for both mag/min filters, performing weighted interpolation between the four nearest texels in a 2D neighborhood. You'll observe smoother texture appearance compared to nearest-neighbour, with reduced blockiness on your road surface, though some blurring may occur at extreme viewing angles where the texture stretches into the distance.

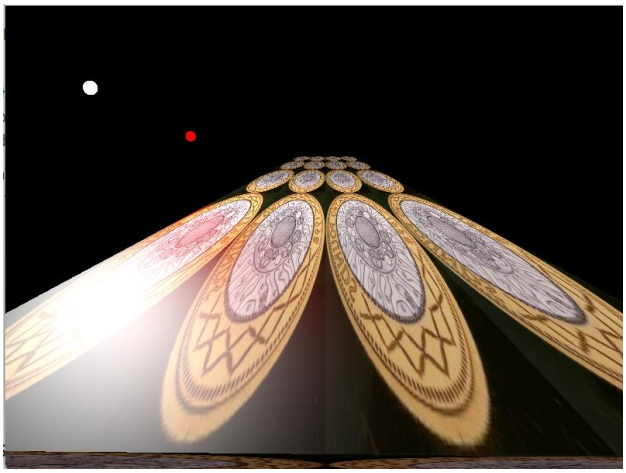
```
void HelloTriangleApplication::createTextureSampler() {
    VkPhysicalDeviceProperties properties{};
    vkGetPhysicalDeviceProperties(physicalDevice, &properties);

    VkSamplerCreateInfo samplerInfo{};
    samplerInfo.sType = VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO;
    samplerInfo.pNext = nullptr;
    samplerInfo.flags = 0;

    // Bilinear filtering: linear interpolation for both mag and min
    samplerInfo.magFilter = VK_FILTER_LINEAR;
    samplerInfo.minFilter = VK_FILTER_LINEAR;

    samplerInfo.addressModeU = VK_SAMPLER_ADDRESS_MODE_REPEAT;
    samplerInfo.addressModeV = VK_SAMPLER_ADDRESS_MODE_REPEAT;
    samplerInfo.addressModeW = VK_SAMPLER_ADDRESS_MODE_REPEAT;

    // No anisotropic filtering for pure bilinear
    samplerInfo.anisotropyEnable = VK_FALSE;
    samplerInfo.maxAnisotropy = 1.0f;

    samplerInfo.borderColor = VK_BORDER_COLOR_INT_OPAQUE_BLACK;
    samplerInfo.unnormalizedCoordinates = VK_FALSE;
    samplerInfo.compareEnable = VK_FALSE;
    samplerInfo.compareOp = VK_COMPARE_OP_ALWAYS;

    // Linear mipmap blending for smoother transitions
    samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_LINEAR;
    samplerInfo.mipLodBias = 0.0f;
    samplerInfo.minLod = 0.0f;
    samplerInfo.maxLod = 0.0f;
```



Trilinear Filtering (Bi-cubic approximation)

Building on bilinear filtering, this adds VK_SAMPLER_MIPMAP_MODE_LINEAR to blend between adjacent mipmap levels, requiring maxLod to be set to VK_LOD_CLAMP_NONE. The road texture will transition smoothly between detail levels as it recedes into the distance, eliminating the visible "popping" artifacts that occur when switching between discrete mipmap levels, though this requires generating mipmaps for your texture during creation.

```
void HelloTriangleApplication::createTextureSampler() {
    VkPhysicalDeviceProperties properties{};
    vkGetPhysicalDeviceProperties(physicalDevice, &properties);

    VkSamplerCreateInfo samplerInfo{};
    samplerInfo.sType = VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO;
    samplerInfo.pNext = nullptr;
    samplerInfo.flags = 0;

    // Trilinear filtering: linear + linear mipmap blending
    samplerInfo.magFilter = VK_FILTER_LINEAR;
    samplerInfo.minFilter = VK_FILTER_LINEAR;

    samplerInfo.addressModeU = VK_SAMPLER_ADDRESS_MODE_REPEAT;
    samplerInfo.addressModeV = VK_SAMPLER_ADDRESS_MODE_REPEAT;
    samplerInfo.addressModeW = VK_SAMPLER_ADDRESS_MODE_REPEAT;

    // Disable anisotropic for pure trilinear
    samplerInfo.anisotropyEnable = VK_FALSE;
    samplerInfo.maxAnisotropy = 1.0f;

    samplerInfo.borderColor = VK_BORDER_COLOR_INT_OPAQUE_BLACK;
    samplerInfo.unnormalizedCoordinates = VK_FALSE;
    samplerInfo.compareEnable = VK_FALSE;
    samplerInfo.compareOp = VK_COMPARE_OP_ALWAYS;

    // KEY: Linear mipmap mode enables trilinear filtering
    samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_LINEAR;
    samplerInfo.mipLodBias = 0.0f;
    samplerInfo.minLod = 0.0f;
    samplerInfo.maxLod = VK_LOD_CLAMP_NONE;  // Allow all mipmap levels
```



Exercise 6:

Question:

Multiple texturing

Solution:

The descriptor layout, pool and sets are updated to support additional textures.

```
void HelloTriangleApplication::createDescriptorSetLayout() {
    // Binding 0: Uniform Buffer Object
    VkDescriptorSetLayoutBinding uboLayoutBinding{};
    uboLayoutBinding.binding = 0;
    uboLayoutBinding.descriptorCount = 1;
    uboLayoutBinding.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
    uboLayoutBinding.stageFlags = VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT;

    // Binding 1: Coin texture (with alpha)
    VkDescriptorSetLayoutBinding coinSamplerLayoutBinding{};
    coinSamplerLayoutBinding.binding = 1;
    coinSamplerLayoutBinding.descriptorCount = 1;
    coinSamplerLayoutBinding.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
    coinSamplerLayoutBinding.stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT;

    // Binding 2: Tiles texture (background)
    VkDescriptorSetLayoutBinding tilesSamplerLayoutBinding{};
    tilesSamplerLayoutBinding.binding = 2;
    tilesSamplerLayoutBinding.descriptorCount = 1;
    tilesSamplerLayoutBinding.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
    tilesSamplerLayoutBinding.stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT;

    std::array<VkDescriptorSetLayoutBinding, 3> bindings = {
        uboLayoutBinding,
        coinSamplerLayoutBinding,
        tilesSamplerLayoutBinding
    };

    VkDescriptorSetLayoutCreateInfo layoutInfo{};
    layoutInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_LAYOUT_CREATE_INFO;
    layoutInfo.bindingCount = static_cast<uint32_t>(bindings.size());
    layoutInfo.pBindings = bindings.data();
```

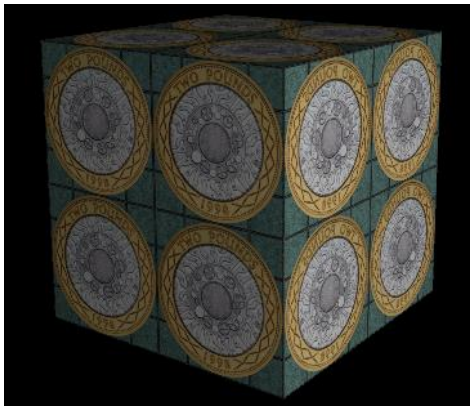
```
void HelloTriangleApplication::createDescriptorPool() {
    std::array<VkDescriptorPoolSize, 2> poolSizes{};
    poolSizes[0].type = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
    poolSizes[0].descriptorCount = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);
    poolSizes[1].type = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
    poolSizes[1].descriptorCount = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT) * 2;

    VkDescriptorPoolCreateInfo poolInfo{};
    poolInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_POOL_CREATE_INFO;
    poolInfo.poolSizeCount = static_cast<uint32_t>(poolSizes.size());
    poolInfo.pPoolSizes = poolSizes.data();
    poolInfo.maxSets = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);
```

The graphics pipeline now supports alpha blending, which allows for changes in texture transparency.

```
VkPipelineColorBlendAttachmentState colorBlendAttachment{};
colorBlendAttachment.colorWriteMask = VK_COLOR_COMPONENT_R_BIT | VK_COLOR_COMPONENT_G_BIT |
    VK_COLOR_COMPONENT_B_BIT | VK_COLOR_COMPONENT_A_BIT;
colorBlendAttachment.blendEnable = VK_TRUE;
colorBlendAttachment.srcColorBlendFactor = VK_BLEND_FACTOR_SRC_ALPHA;
colorBlendAttachment.dstColorBlendFactor = VK_BLEND_FACTOR_ONE_MINUS_SRC_ALPHA;
colorBlendAttachment.colorBlendOp = VK_BLEND_OP_ADD;
colorBlendAttachment.srcAlphaBlendFactor = VK_BLEND_FACTOR_ONE;
colorBlendAttachment.dstAlphaBlendFactor = VK_BLEND_FACTOR_ZERO;
colorBlendAttachment.alphaBlendOp = VK_BLEND_OP_ADD;
```

Using the mix() GLSL function, the textures can be blended using linear interpolation, after removing the background of the coin with an external tool.

```
// Sample both textures
vec4 coinColor = texture(coinSampler, vTexCoord);
vec4 tilesColor = texture(tilesSampler, vTexCoord);

// Blend: use coin where alpha > 0, otherwise use tiles
vec4 texColor = mix(tilesColor, coinColor, coinColor.a);
```



Exercise 7:

Question:

AN OPEN BOX

Solution:

The CreateOpenBox() function creates a 5-faced cube, with each face duplicated for interior and exterior.

```
GeometryGenerator::MeshData GeometryGenerator::CreateOpenBox(float width, float height, float depth) {
    MeshData meshData;

    float w2 = width * 0.5f;
    float h2 = height * 0.5f;
    float d2 = depth * 0.5f;

    // Reserve space for vertices (5 faces * 2 sides * 4 vertices = 40 vertices)
    meshData.vertices.reserve(40);

    // IMPORTANT: Face order is INTERIOR first (odd indices), then EXTERIOR (even indices)
    // to match the material assignment logic: even = rocks (exterior), odd = wood (interior)

    // --- FRONT FACE ---
    // Face 0: INTERIOR (wood - materialId 1)
    meshData.vertices.push_back(Vertex{ { w2, -h2, d2}, {0.0f, 0.0f, -1.0f}, {1.0f, 1.0f, 1.0f}, {0.0f, 0.0f} });
    meshData.vertices.push_back(Vertex{ {-w2, -h2, d2}, {0.0f, 0.0f, -1.0f}, {1.0f, 1.0f, 1.0f}, {1.0f, 0.0f} });
    meshData.vertices.push_back(Vertex{ {-w2, h2, d2}, {0.0f, 0.0f, -1.0f}, {1.0f, 1.0f, 1.0f}, {1.0f, 1.0f} });
    meshData.vertices.push_back(Vertex{ { w2, h2, d2}, {0.0f, 0.0f, -1.0f}, {1.0f, 1.0f, 1.0f}, {0.0f, 1.0f} });

    // Face 1: EXTERIOR (rocks - materialId 2)
    meshData.vertices.push_back(Vertex{ {-w2, -h2, d2}, {0.0f, 0.0f, 1.0f}, {1.0f, 1.0f, 1.0f}, {0.0f, 0.0f} });
    meshData.vertices.push_back(Vertex{ { w2, -h2, d2}, {0.0f, 0.0f, 1.0f}, {1.0f, 1.0f, 1.0f}, {1.0f, 0.0f} });
    meshData.vertices.push_back(Vertex{ { w2, h2, d2}, {0.0f, 0.0f, 1.0f}, {1.0f, 1.0f, 1.0f}, {1.0f, 1.0f} });
    meshData.vertices.push_back(Vertex{ {-w2, h2, d2}, {0.0f, 0.0f, 1.0f}, {1.0f, 1.0f, 1.0f}, {0.0f, 1.0f} });
```

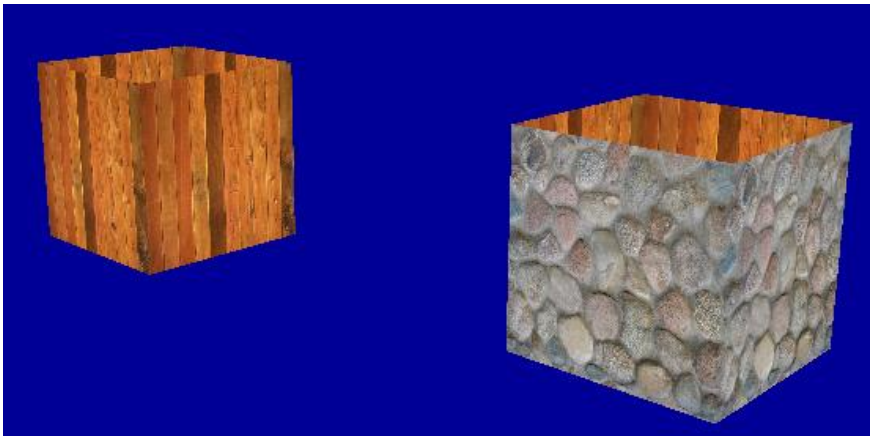
Depending on if the face is an even or odd number, a different texture can be assigned.

```
vertexOffset = static_cast<uint32_t>(vertices.size());
indexOffset = static_cast<uint32_t>(indices.size());
vertices.insert(vertices.end(), woodBox.vertices.begin(), woodBox.vertices.end());
indices.insert(indices.end(), woodBox.indices.begin(), woodBox.indices.end());

// Draw exterior faces (even indices 0, 2, 4, 6, 8) with rocks
// Draw interior faces (odd indices 1, 3, 5, 7, 9) with wood
for (uint32_t face = 0; face < 10; ++face) {
    bool isExterior = (face % 2 == 0);

    drawCommands.push_back({
        6,  // 2 triangles = 6 indices per face
        indexOffset + (face * 6),
        static_cast<int32_t>(vertexOffset),
        glm::translate(glm::mat4(1.0f), glm::vec3(3.0f, 1.0f, 0.0f)), // Changed Y from 0.0 to 1.0
        true,   // shouldRotate
        false,
        isExterior ? 2 : 1,  // materialId: 2 for rocks, 1 for wood
        0       // lightId
        });
}
```

The material ID will be used to determine the texture ID, which decides assignment in the fragment shader.

```
switch (cmd.materialId) {
case 1:
    // Wood material
    pc.ambient = glm::vec3(0.5f);
    pc.diffuse = glm::vec3(0.5f);
    pc.specular = glm::vec3(0.3f);
    pc.shininess = 16.0f;
    pc.textureId = 0;  // Wood
    break;
case 2:
    // Rocks material
    pc.ambient = glm::vec3(0.5f);
    pc.diffuse = glm::vec3(0.5f);
    pc.specular = glm::vec3(0.5f);
    pc.shininess = 32.0f;
    pc.textureId = 1;  // Rocks
    break;
```

```
vec4 texColor = (pc.textureId == 0) ? texture(woodSampler, vTexCoord) : texture(rocksSampler, vTexCoord);
```



Lab 6

Date: 11/11/2025

Exercise 1+2:

Question:

PREPARING FOR TANGENT SPACE + NORMAL MAPPING IMPLEMENTATION

Solution:

Vertex.h updated to include tangent and binormal.

```
struct Vertex {
    glm::vec3 pos;
    glm::vec3 color;
    glm::vec3 normal;
    glm::vec2 texCoord;
    glm::vec3 tangent;   // New
    glm::vec3 binormal;  // New

    static VkVertexInputBindingDescription getBindingDescription() {
        VkVertexInputBindingDescription bindingDescription{};
        bindingDescription.binding = 0;
        bindingDescription.stride = sizeof(Vertex);
        bindingDescription.inputRate = VK_VERTEX_INPUT_RATE_VERTEX;
        return bindingDescription;
    }

    static std::array<VkVertexInputAttributeDescription, 6> getAttributeDescriptions() {
        std::array<VkVertexInputAttributeDescription, 6> attributeDescriptions{};
        attributeDescriptions[0] = { 0, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, pos) };
        attributeDescriptions[1] = { 1, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, color) };
        attributeDescriptions[2] = { 2, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, normal) };
        attributeDescriptions[3] = { 3, 0, VK_FORMAT_R32G32_SFLOAT,  offsetof(Vertex, texCoord) };
        attributeDescriptions[4] = { 4, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, tangent) };
        attributeDescriptions[5] = { 5, 0, VK_FORMAT_R32G32B32_SFLOAT, offsetof(Vertex, binormal) };
```

Changes in Vertex struct reflected in object creation

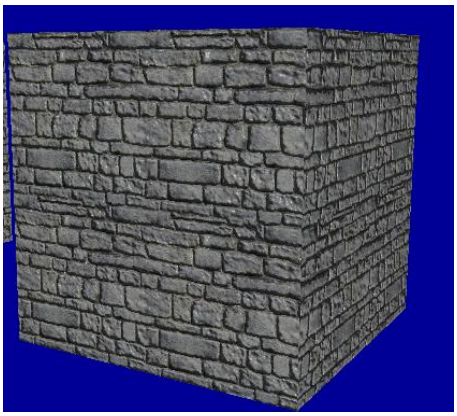
```
glm::vec3 frontNormal = { 0.0f, 0.0f, 1.0f };
glm::vec3 frontTangent = { 1.0f, 0.0f, 0.0f };  // U-dir: -w to +w = +X
glm::vec3 frontBinormal = { 0.0f, 1.0f, 0.0f }; // V-dir: -h to +h = +Y
glm::vec3 frontColor = { 1.0f, 0.0f, 0.0f }; // Red

meshData.vertices.push_back(Vertex{ {-w, -h,  d}, frontColor, frontNormal, {0.0f, 2.0f}, frontTangent, frontBinormal }); // BL
meshData.vertices.push_back(Vertex{ { w,  h,  d}, frontColor, frontNormal, {2.0f, 0.0f}, frontTangent, frontBinormal }); // TR
meshData.vertices.push_back(Vertex{ { w, -h,  d}, frontColor, frontNormal, {2.0f, 2.0f}, frontTangent, frontBinormal }); // BR

meshData.vertices.push_back(Vertex{ {-w, -h,  d}, frontColor, frontNormal, {0.0f, 2.0f}, frontTangent, frontBinormal }); // BL
meshData.vertices.push_back(Vertex{ {-w,  h,  d}, frontColor, frontNormal, {0.0f, 0.0f}, frontTangent, frontBinormal }); // TL
meshData.vertices.push_back(Vertex{ { w,  h,  d}, frontColor, frontNormal, {2.0f, 0.0f}, frontTangent, frontBinormal }); // TR
```

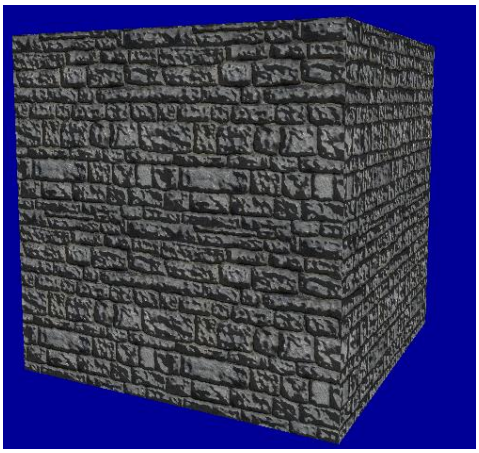
The VkFormat was changed from SRGB to UNORM, and the lighting parameters were adjusted, lowering ambience and increasing brightness.

The vertex shader calculates the TBN matrix and transforms the light and view positions into tangent space, passing them to the fragment shader

```
void main() {
    // --- Standard MVP Transform for gl_Position ---
    mat4 view = lookAt(ubo.eyePosition, ubo.lookAtPoint, ubo.upDirection);
    mat4 proj = perspective(ubo.fovy, ubo.aspect, ubo.zNear, ubo.zFar);
    gl_Position = proj * view * pc.model * vec4(inPosition, 1.0);

    // Pass texture coordinates
    fragTexCoord = inTexCoord;

    // --- TBN Matrix Calculation ---
    // Note: Using pc.model, not ubo.model as in exercise text
    // This matrix transforms normals from model to world space
    mat3 ModelMatrix_TInv = mat3(transpose(inverse(pc.model)));

    // Transform T, B, N vectors to world space
    vec3 T = normalize(ModelMatrix_TInv * inTangent);
    vec3 B = normalize(ModelMatrix_TInv * inBinormal);
    vec3 N = normalize(ModelMatrix_TInv * inNormal);

    // Create the TBN matrix (transforms from world-space to tangent-space)
    // This is the inverse of the matrix [T B N]
    mat3 TBN = transpose(mat3(T, B, N));

    // Get world-space light and view positions
    vec3 lightPos_world = ubo.lightPos;
    vec3 viewPos_world = ubo.eyePosition; // Use eyePosition from UBO
    vec3 fragPos_world = (pc.model * vec4(inPosition, 1.0)).xyz;

    // Transform positions to tangent space and send to fragment shader
    fragLightPos_tangent = TBN * lightPos_world;
    fragViewPos_tangent = TBN * viewPos_world;
    fragPos_tangent = TBN * fragPos_world;
}
```

Tangent-space lighting logic is added to the fragment shader.





N_tangent.xy *=10.0

```
void main() {
     // Get base color
    vec3 albedo = texture(colSampler, fragTexCoord).rgb;

    // Get normal from normal map
    // .rgb stores X, Y, Z components of the normal vector
    // We "unpack" it from the [0, 1] texture range to the [-1, 1] vector range

    vec3 N_tangent = texture(normalSampler, fragTexCoord).rgb;
    N_tangent = normalize(N_tangent * 2.0 - 1.0);

    // Increase normal map effect
    N_tangent.xy *= 2.5;
    N_tangent = normalize(N_tangent);

    // --- Implement lighting in tangent space ---
    // All vectors are now in the same (tangent) space

    // N is from the normal map
    vec3 N = N_tangent;

    // L is the vector from fragment to light
    vec3 L = normalize(fragLightPos_tangent - fragPos_tangent);

    // V is the vector from fragment to view
    vec3 V = normalize(fragViewPos_tangent - fragPos_tangent);

    // R is the reflection vector
    vec3 R = reflect(-L, N);

    // --- Compute Blinn-Phong (for the "white" light) ---
    float diff = max(dot(N, L), 0.0);

    float spec = 0.0;
    if (diff > 0.0) {
        // Use Blinn-Phong specular (dot(R, V))
        spec = pow(max(dot(R, V), 0.0), pc.shininess);
    }

    // Using the "white" light from your UBO (ubo.lightColor)
    vec3 ambient = pc.ambient * albedo * ubo.lightColor;
    vec3 diffuse = pc.diffuse * albedo * diff * ubo.lightColor;
    vec3 specular = pc.specular * spec * ubo.lightColor;

    // Final color
    vec3 result = ambient + diffuse + specular;
    outColor = vec4(result, 1.0);
}
```

Exercise 3:

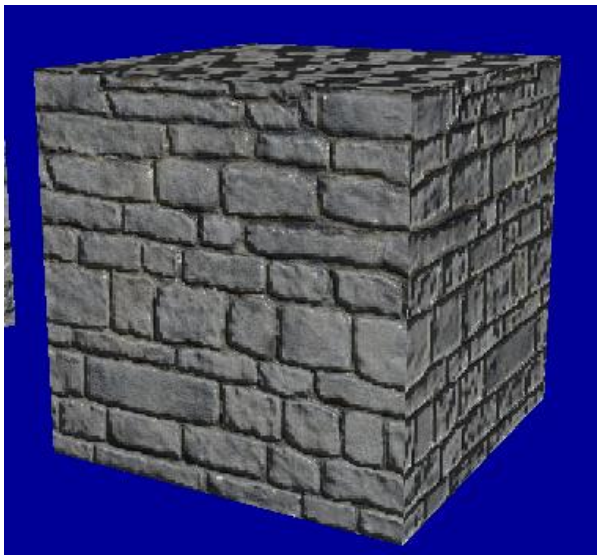
Question:

HEIGHT MAP BUMPY EFFECT

Solution:

A bump mapping effect was implemented using only a grayscale height map instead of a normal map. In the fragment shader, the height map's value was sampled, and then the dFdx and dFdy functions were used to calculate the derivatives, or slope, of the height at that fragment. This slope information was then used to mathematically construct a new tangent-space normal vector, which was passed to the existing lighting code to create the illusion of a bumpy surface.

```
void main() {
     // Get base color
    vec3 albedo = texture(colSampler, fragTexCoord).rgb;

    // 1. Sample the height (grayscale, so .r is fine)
    float h = texture(heightSampler, fragTexCoord).r;

    // 2. Calculate derivatives (slope) of the height
    float dFx = dFdx(h);
    float dFy = dFdy(h);

    // 3. Set the bump intensity.
    // A smaller value (e.g., 0.1) makes the derivatives
    // more significant, increasing the bumpy effect.
    float bumpHeight = 0.1;

    // 4. Create the new tangent-space normal
    vec3 N_tangent = normalize(vec3(-dFx, -dFy, bumpHeight));

    // --- Implement lighting in tangent space ---
    // All vectors are now in the same (tangent) space

    // N is from the height map calculation
    vec3 N = N_tangent;
    // L is the vector from fragment to light
    vec3 L = normalize(fragLightPos_tangent - fragPos_tangent);
    // V is the vector from fragment to view
    vec3 V = normalize(fragViewPos_tangent - fragPos_tangent);
    // R is the reflection vector
    vec3 R = reflect(-L, N);

    // --- Compute Blinn-Phong (for the "white" light) ---
    float diff = max(dot(N, L), 0.0);
    float spec = 0.0;
    if (diff > 0.0) {
        // Use Blinn-Phong specular (dot(R, V))
        spec = pow(max(dot(R, V), 0.0), pc.shininess);
    }

    // Using the "white" light from your UBO (ubo.lightColor)
    vec3 ambient = pc.ambient * albedo * ubo.lightColor;
    vec3 diffuse = pc.diffuse * albedo * diff * ubo.lightColor;
    vec3 specular = pc.specular * spec * ubo.lightColor;

    // Final color
    vec3 result = ambient + diffuse + specular;
    outColor = vec4(result, 1.0);
}
```



Exercise 4:

Question:

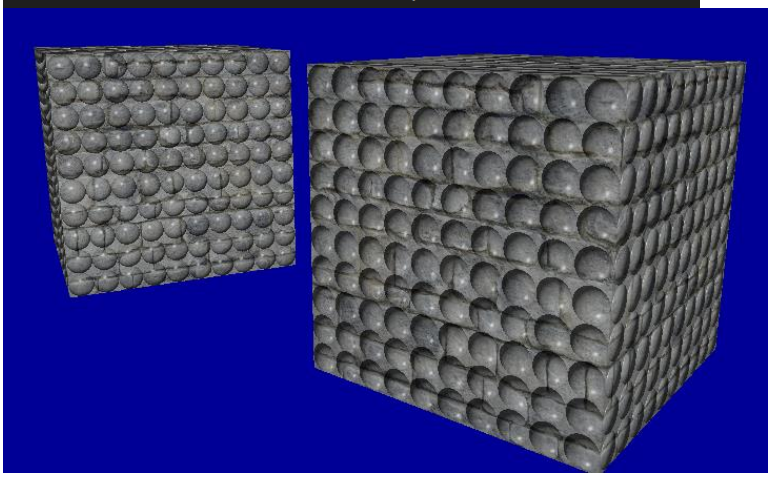
PROCEDURAL NORMAL MAPPING

Solution:

A procedural bump mapping effect was implemented in the fragment shader to create a bumpy surface without using any texture maps. A new bumpNormal function was added, which uses mathematical logic and the fract() function to procedurally calculate a new tangent-space normal vector simulating a pattern of hemispheres. This "invented" normal was then passed directly to the existing Blinn-Phong lighting calculations, which lit the synthetic, patterned surface as if it were a real 3D shape

```
vec3 bumpNormal(vec2 xy) {
    // Default normal is flat (pointing up in tangent space)
    vec3 N = vec3(0.0, 0.0, 1.0);

    // Define the radius of the procedural hemisphere
    // set it to 1.0 to tile perfectly with the fract() function
    float radius = 1.0;

    // map xy to [-1, 1]x[-1, 1] using fract() for tiling
    vec2 st = 2.0 * fract(xy) - 1.0;

    // Calculate R-squared minus distance-squared (z-squared of a hemisphere)
    float R2 = radius * radius - dot(st, st);

    // If we are inside the bump's radius
    if (R2 > 0.0) {
        // Calculate the normal's XY components
        N.xy = st / sqrt(R2);
        return normalize(N);
    }

    // If outside the radius (in the "flat" area), return the default flat normal
    return N;
}
```

```
// 1. Define the tiling density for the pattern
float BumpDensity = 10.0;

// 2. Calculate the tiled coordinates to pass to the bump function
// (using fragTexCoord, which is the 'input.Tex' from the example)
vec2 bumpCoords = fragTexCoord * BumpDensity;

// 3. Generate the procedural normal in tangent space
vec3 N_tangent = bumpNormal(bumpCoords);
```



Exercise 5:

Question:

RAY MARCHING -BASED PARALLAX MAPPING

Solution:

The descriptor pipeline was updated to manage three samplers – colour, normal and height.

The descriptor set layout expects 4 bindings, the descriptor pool size was increased to allocate 3 samplers per frame and the descriptor set now has a height texture bound to 3.

```
VkDescriptorBufferInfo bufferInfo{};
bufferInfo.buffer = uniformBuffers[i];
bufferInfo.offset = 0;
bufferInfo.range = sizeof(UniformBufferObject);

VkDescriptorImageInfo colourImageInfo{};
colourImageInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
colourImageInfo.imageView = colourTexture.imageView;
colourImageInfo.sampler = colourTexture.sampler;

VkDescriptorImageInfo normalImageInfo{};
normalImageInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
normalImageInfo.imageView = normalTexture.imageView;
normalImageInfo.sampler = normalTexture.sampler;

VkDescriptorImageInfo heightImageInfo{};
heightImageInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
heightImageInfo.imageView = heightTexture.imageView;
heightImageInfo.sampler = heightTexture.sampler;

std::array<VkWriteDescriptorSet, 4> descriptorWrites{};
```

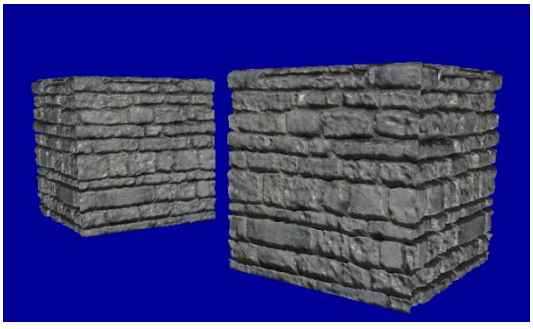
```
void HelloTriangleApplication::createDescriptorSetLayout() {
    // Binding 0: Uniform Buffer Object
    VkDescriptorSetLayoutBinding uboLayoutBinding{};
    uboLayoutBinding.binding = 0;
    uboLayoutBinding.descriptorCount = 1;
    uboLayoutBinding.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
    uboLayoutBinding.stageFlags = VK_SHADER_STAGE_VERTEX_BIT | VK_SHADER_STAGE_FRAGMENT_BIT

    // Binding 1: Colour texture
    VkDescriptorSetLayoutBinding colourSamplerLayoutBinding{};
    colourSamplerLayoutBinding.binding = 1;
    colourSamplerLayoutBinding.descriptorCount = 1;
    colourSamplerLayoutBinding.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
    colourSamplerLayoutBinding.stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT;

    // Binding 2: Normal texture
    VkDescriptorSetLayoutBinding normalSamplerLayoutBinding{};
    normalSamplerLayoutBinding.binding = 2;
    normalSamplerLayoutBinding.descriptorCount = 1;
    normalSamplerLayoutBinding.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
    normalSamplerLayoutBinding.stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT;

    // New: Binding 3: Height texture
    VkDescriptorSetLayoutBinding heightSamplerLayoutBinding{};
    heightSamplerLayoutBinding.binding = 3;
    heightSamplerLayoutBinding.descriptorCount = 1;
    heightSamplerLayoutBinding.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
    heightSamplerLayoutBinding.stageFlags = VK_SHADER_STAGE_FRAGMENT_BIT;

    std::array<VkDescriptorSetLayoutBinding, 4> bindings = {
        uboLayoutBinding,
        colourSamplerLayoutBinding,
        normalSamplerLayoutBinding,
        heightSamplerLayoutBinding
    };
```

The fragment shader accepts the new texture and a Ray Marching function was added, which takes the fragments original texture coordinate and the tangent-space view direction. It steps iteratively into the surface, sampling the heightSampler at each step to see if the ray has collided with the terrain.

This function ins called in main to get the displaced UVs, which is used to sample the colour and normal sampler. This creates the 3D effect.

```
void main() {
    vec3 V = normalize(fragViewPos_tangent - fragPos_tangent);
    vec2 displacedUVs = parallaxRayMarch(fragTexCoord, V);

    // sample at displaced coords
    vec3 albedo = texture(colourSampler, displacedUVs).rgb;
    vec3 N_tangent = texture(normalSampler, displacedUVs).rgb;
    N_tangent = normalize(N_tangent * 2.0 - 1.0);

    vec3 N = N_tangent;
    vec3 L = normalize(fragLightPos_tangent - fragPos_tangent);
    vec3 R = reflect(-L, N);

    float diff = max(dot(N, L), 0.0);
    float spec = 0.0;
    if (diff > 0.0) {
        spec = pow(max(dot(R, V), 0.0), pc.shininess);
    }

    vec3 ambient = pc.ambient * albedo * ubo.lightColor;
    vec3 diffuse = pc.diffuse * albedo * diff * ubo.lightColor;
    vec3 specular = pc.specular * spec * ubo.lightColor;

    vec3 result = ambient + diffuse + specular;

    // discard out-of-range displaced UVs
    if (displacedUVs.x < 0.0 || displacedUVs.x > 1.0 || displacedUVs.y < 0.0 || displacedUVs.y > 1.0) {
        discard;
    }

    outColor = vec4(result, 1.0);
}
```

```
vec2 parallaxRayMarch(vec2 texCoords, vec3 viewDir_tangent) {
    const int numSteps = 20;
    float heightScale = 0.05;
    float currentRayHeight = 1.0;
    float stepZ = 1.0 / float(numSteps);
    vec2 uvStep = viewDir_tangent.xy * heightScale / viewDir_tangent.z / float(numSteps)
    vec2 currentUV = texCoords;

    for (int i = 0; i < numSteps; i++) {
        float heightMapValue = texture(heightSampler, currentUV).r;
        if (currentRayHeight < heightMapValue) {
            vec2 prevUV = currentUV + uvStep;
            float prevRayHeight = currentRayHeight + stepZ;
            float distToPrev = (prevRayHeight - texture(heightSampler, prevUV).r);
            float distToCurr = (heightMapValue - currentRayHeight);
            float weight = distToCurr / (distToCurr + distToPrev);
            return mix(currentUV, prevUV, weight);
        }
        currentUV -= uvStep;
        currentRayHeight -= stepZ;
    }
    return currentUV;
}
```



Lab 7

Date: 14/11/2025

Exercise 2:

Question:

IMPLEMENTING THE SKYBOX

Solution:

Depth buffer required in Ex1 was implemented in Lab 4.

Skybox fragment and vertex shaders were created and used a skybox pipeline to handle rendering.

Skybox.frag

```
#version 450

layout(location = 1) in vec3 viewDir;

// Use binding 3 for the skybox sampler (to avoid conflict with existing textures)
layout(binding = 3) uniform samplerCube skySampler;

layout(location = 0) out vec4 outColor;

void main() {
    // Sample the cubemap using the vertex position as a direction vector
    outColor = texture(skySampler, viewDir);
}
```

Skybox.vert

```
void main() {
    viewDir = inPosition;

    // 1. Get View Matrix
    mat4 view = lookAt(ubo.eyePosition, ubo.lookAtPoint, ubo.upDirection);

    // 2. Remove translation to keep skybox centered on camera
    view = mat4(mat3(view));

    // 3. Get Projection
    mat4 proj = perspective(ubo.fovy, ubo.aspect, ubo.zNear, ubo.zFar);

    // 4. Apply transformation
    vec4 pos = proj * view * vec4(inPosition, 1.0);

    // Optimization: Force z to w so that z/w = 1.0 (farthest depth)
    gl_Position = pos.xyww;
}
```

createSkyboxPipeline() follows a similar setup to the main graphics pipeline. Depth and culling are handled differently, as to ensure other geometry isn't interfered with.

```
void HelloTriangleApplication::createSkyboxPipeline() {
    auto vertShaderCode = readFile("shaders/skybox_vert.spv");
    auto fragShaderCode = readFile("shaders/skybox_frag.spv");

    VkShaderModule vertShaderModule = createShaderModule(vertShaderCode);
    VkShaderModule fragShaderModule = createShaderModule(fragShaderCode);

    VkPipelineShaderStageCreateInfo vertShaderStageInfo
      { VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO };
    vertShaderStageInfo.stage = VK_SHADER_STAGE_VERTEX_BIT;
    vertShaderStageInfo.module = vertShaderModule;
    vertShaderStageInfo.pName = "main";

    VkPipelineShaderStageCreateInfo fragShaderStageInfo
      { VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO };
    fragShaderStageInfo.stage = VK_SHADER_STAGE_FRAGMENT_BIT;
    fragShaderStageInfo.module = fragShaderModule;
    fragShaderStageInfo.pName = "main";

    VkPipelineShaderStageCreateInfo shaderStages[] = { vertShaderStageInfo,
      fragShaderStageInfo };

    // Use existing Vertex description (we will render a cube)
    auto bindingDescription = Vertex::getBindingDescription();
    auto attributeDescriptions = Vertex::getAttributeDescriptions();
```

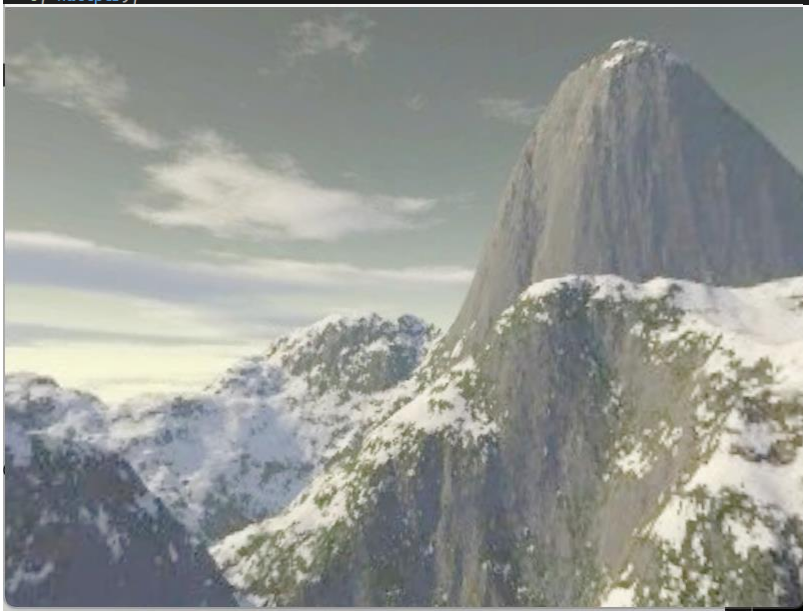
```
VkPipelineRasterizationStateCreateInfo rasterizer
  { VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO };
rasterizer.polygonMode = VK_POLYGON_MODE_FILL;
rasterizer.lineWidth = 1.0f;
rasterizer.cullMode = VK_CULL_MODE_FRONT_BIT;
rasterizer.frontFace = VK_FRONT_FACE_CLOCKWISE;

VkPipelineMultisampleStateCreateInfo multisampling
  { VK_STRUCTURE_TYPE_PIPELINE_MULTISAMPLE_STATE_CREATE_INFO };
multisampling.rasterizationSamples = VK_SAMPLE_COUNT_1_BIT;

// --- KEY CHANGE: Depth Stencil ---
VkPipelineDepthStencilStateCreateInfo depthStencil
  { VK_STRUCTURE_TYPE_PIPELINE_DEPTH_STENCIL_STATE_CREATE_INFO };
depthStencil.depthTestEnable = VK_TRUE;
depthStencil.depthWriteEnable = VK_FALSE; // Don't overwrite depth
depthStencil.depthCompareOp = VK_COMPARE_OP_LESS_OR_EQUAL; // Allow drawing at depth 1.0
```

loadSkybox() creates a skybox image with 6 array layers and a VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT flag, allowing the GPU to treat the 6 layers as a single cube object. The skyboxImageView is created with viewType = VK_IMAGE_VIEW_TYPE_CUBE and a layerCount = 6, ensuring the shaders can sample it correctly as a 3D direction vector. A skyboxSampler was created with VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE to prevent edge artifacts between the cube faces.

```cpp
void HelloTriangleApplication::loadSkybox() {
    // Order defined by GLSL/Vulkan spec: +X, -X, +Y, -Y, +Z, -Z
    std::vector<std::string> filenames = {
        "cubemap_0(+X).jpg", "cubemap_1(-X).jpg",
        "cubemap_2(+Y).jpg", "cubemap_3(-Y).jpg",
        "cubemap_4(+Z).jpg", "cubemap_5(-Z).jpg"
    };

    int texWidth, texHeight, texChannels;
    stbi_uc* pixels[6];

    // 1. Load all 6 images
    for (int i = 0; i < 6; i++) {
        pixels[i] = stbi_load(filenames[i].c_str(), &texWidth, &texHeight, &texChannels, STBI_rgb_alpha);
        if (!pixels[i]) throw std::runtime_error("Failed to load skybox image: " + filenames[i]);
    }

    VkDeviceSize layerSize = texWidth * texHeight * 4;
    VkDeviceSize imageSize = layerSize * 6;

    // 2. Create Staging Buffer
    VkBuffer stagingBuffer;
    VkDeviceMemory stagingBufferMemory;
    createBuffer(imageSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT, VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
      VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, stagingBuffer, stagingBufferMemory);

    // 3. Copy data to staging buffer
    void* data;
    vkMapMemory(device, stagingBufferMemory, 0, imageSize, 0, &data);
    for (int i = 0; i < 6; i++) {
        memcpy(static_cast<char*>(data) + (layerSize * i), pixels[i], static_cast<size_t>(layerSize));
        stbi_image_free(pixels[i]);
    }
    vkUnmapMemory(device, stagingBufferMemory);

    // 4. Create GPU Image (Cubemap Compatible)
    createImage(texWidth, texHeight, 6, VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT, VK_FORMAT_R8G8B8A8_UNORM,
      VK_IMAGE_TILING_OPTIMAL, VK_IMAGE_USAGE_TRANSFER_DST_BIT | VK_IMAGE_USAGE_SAMPLED_BIT,
      VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, skyboxImage, skyboxImageMemory);

    // 5. Transition and Copy
    transitionImageLayout(skyboxImage, VK_FORMAT_R8G8B8A8_UNORM, VK_IMAGE_LAYOUT_UNDEFINED,
      VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL); // Note: Update transition helper to handle layerCount=6 if
```

Make sure skybox pipeline is built before graphics pipeline in recordCommandBuffer()

```cpp
vkCmdBeginRendering(commandBuffer, &renderingInfo);

// 1. Draw Skybox first
vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, skyboxPipeline);
vkCmdBindDescriptorSets(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, skyboxPipelineLayout, 0, 1, &descriptorSets
  [currentFrame], 0, nullptr);

// Bind the cube mesh (reuse the box created in initVulkan)
VkBuffer vertexBuffers[] = { vertexBuffer };
VkDeviceSize offsets[] = { 0 };
vkCmdBindVertexBuffers(commandBuffer, 0, 1, vertexBuffers, offsets);
vkCmdBindIndexBuffer(commandBuffer, indexBuffer, 0, VK_INDEX_TYPE_UINT32);

// Issue draw call for the cube (first 36 indices if the cube is the first thing in your buffer)
vkCmdDrawIndexed(commandBuffer, 36, 1, 0, 0, 0);

// 2. Draw Scene objects (bind main pipeline, etc.)

vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, graphicsPipeline);
vkCmdBindDescriptorSets(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, pipelineLayout, 0, 1, &descriptorSets[currentFrame],
  0, nullptr);
```



Exercise 3 + 4:

Question:

IMPLEMENTING REFLECTIONS & REFRACTIONS

Solution:

Shader.vert

```
void main() {
    mat4 view = lookAt(ubo.eyePosition, ubo.lookAtPoint, ubo.upDirection);
    mat4 proj = perspective(ubo.fovy, ubo.aspect, ubo.zNear, ubo.zFar);
    gl_Position = proj * view * pc.model * vec4(inPosition, 1.0);

    fragTexCoord = inTexCoord;

    // 1. Calculate World Position
    vec4 worldPos = pc.model * vec4(inPosition, 1.0);
    outWorldPos = worldPos.xyz;

    // 2. Calculate World Normal
    // Use the inverse transpose of the model matrix to handle non-uniform scaling correctly
    mat3 normalMatrix = mat3(transpose(inverse(pc.model)));
    outWorldNormal = normalize(normalMatrix * inNormal);

    // --- Existing Tangent Space Logic (Keep this if you want to mix bump mapping later) ---
    mat3 TBN = transpose(mat3(
        normalize(normalMatrix * inTangent),
        normalize(normalMatrix * inBinormal),
        normalize(normalMatrix * inNormal)
    ));

    fragLightPos_tangent = TBN * ubo.lightPos;
    fragViewPos_tangent = TBN * ubo.eyePosition;
    fragPos_tangent = TBN * worldPos.xyz;
}
```

Shader.frag

```
void main() {
    // 1. Normalize the interpolated normal and view direction
    vec3 N = normalize(inWorldNormal);
    vec3 V = normalize(ubo.eyePosition - inWorldPos); // Vector from Surface to Camera

    // 2. Calculate the reflection vector
    // reflect(I, N) expects the Incident vector 'I' (Camera to Surface), so we use -V
    vec3 R = reflect(-V, N);

    // 3. Sample the skybox using the reflection vector
    vec3 reflectionColor = texture(skySampler, R).rgb;

    // 4. Output the reflection
    // You can mix this with the base texture if you want a "glossy" look:
    // vec3 baseColor = texture(colSampler, fragTexCoord).rgb;
    // outColor = vec4(mix(baseColor, reflectionColor, 0.8), 1.0);

    // For Exercise 3 (Perfect Mirror):
    outColor = vec4(reflectionColor, 1.0);
}
```

The vertex shader passes out world space data to the fragment shader, as reflections are calculated using world coordinates rather than the tangent space coordinates used for bump mapping.

The fragment shader uses the reflect() function to find the direction the light bounces of a surface. Changing this to refract() and passing in an index of refraction value produces the image on the right

Instead of calculating Blinn-Phong lighting, the environment was sampled using:

outColor = vec4(texture(skySampler, R).rgb, 1.0);

Exercise 4:

Question:

SPRITE-BASED PARTICLE SYSTEM

Solution:

Particle.vert

```
// Physics Constants
#define particleSpeed 0.2
#define particleSpread 1.75
#define particleSize 0.8
#define particleSystemHeight 5.0

void main() {
    // 1. Calculate View and Projection Manually (since they aren't in the UBO)
    mat4 view = lookAt(ubo.eyePosition, ubo.lookAtPoint, ubo.upDirection);
    mat4 proj = perspective(ubo.fovy, ubo.aspect, ubo.zNear, ubo.zFar);

    // 2. Calculate Life Cycle
    float t = fract(inPosition.z + particleSpeed * ubo.time);
    fragLife = t;

    // 3. Calculate "Physics" Center Position
    vec3 centerPos;
    centerPos.y = particleSystemHeight * t - (particleSystemHeight / 2.0);
    centerPos.x = particleSpread * t * cos(50.0 * inPosition.z);
    centerPos.z = particleSpread * t * sin(120.0 * inPosition.z);

    // 4. Billboarding Logic
    // Extract Camera Right and Up vectors from the View Matrix
    // Row 0 is Right, Row 1 is Up (because view matrix is an inverse transform)
    vec3 cameraRight = vec3(view[0][0], view[1][0], view[2][0]);
    vec3 cameraUp    = vec3(view[0][1], view[1][1], view[2][1]);

    // Calculate final vertex position
    // We use inPosition.x/y (-1 to 1) to offset from the particle center along camera axes
    vec3 worldPos = centerPos
                  + (cameraRight * inPosition.x * particleSize)
                  + (cameraUp * inPosition.y * particleSize);

    gl_Position = proj * view * pc.model * vec4(worldPos, 1.0);

    fragTexCoord = inPosition.xy;
}
```

Particle.frag

```
void main() {
    // 1. Shape Logic (Keep this)
    float dist = length(fragTexCoord);
    if (dist > 1.0) discard;

    // 2. Alpha Logic (Keep this)
    float alpha = (1.0 - dist) * (1.0 - fragLife);

    // 3. Multi-Stage Color Logic (UPDATED)

    // Define your three key colors
    vec3 colorStart  = vec3(1.0, 1.0, 1.0); // White (Hot center)
    vec3 colorMid    = vec3(0.7, 0.6, 0.0); // Orange/Yellow (Main body)
    vec3 colorEnd    = vec3(0.7, 0.0, 0.0); // Dark Red/Black (Smoke/Cooling)

    vec3 finalColor;

    // Define the "Transition Point".
    // 0.1 means the white part only lasts for the bottom 10% of the life.
    float threshold = 0.005;

    if (fragLife < threshold) {
        // STAGE 1: Bottom (White -> Orange)
        // We must re-map fragLife from [0.0 ... 0.1] to [0.0 ... 1.0] for the mix function
        float t = fragLife / threshold;
        finalColor = mix(colorStart, colorMid, t);
    }
    else {
        // STAGE 2: Middle to Top (Orange -> Dark Red)
        // We must re-map fragLife from [0.1 ... 1.0] to [0.0 ... 1.0]
        float t = (fragLife - threshold) / (1.0 - threshold);
        finalColor = mix(colorMid, colorEnd, t);
    }

    outColor = vec4(finalColor, alpha);
```

createParticlePipeline()

```
// 2. Vertex Input (Re-use the standard Vertex struct)
auto bindingDescription = Vertex::getBindingDescription();
auto attributeDescriptions = Vertex::getAttributeDescriptions();

VkPipelineVertexInputStateCreateInfo vertexInputInfo{};
vertexInputInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO;
vertexInputInfo.vertexBindingDescriptionCount = 1;
vertexInputInfo.pVertexBindingDescriptions = &bindingDescription;
vertexInputInfo.vertexAttributeDescriptionCount = static_cast<uint32_t>(attributeDescriptions.size());
vertexInputInfo.pVertexAttributeDescriptions = attributeDescriptions.data();

// 3. Input Assembly
VkPipelineInputAssemblyStateCreateInfo inputAssembly{};
inputAssembly.sType = VK_STRUCTURE_TYPE_PIPELINE_INPUT_ASSEMBLY_STATE_CREATE_INFO;
inputAssembly.topology = VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST;
inputAssembly.primitiveRestartEnable = VK_FALSE;

// 4. Viewport & Scissor (Dynamic)
VkPipelineViewportStateCreateInfo viewportState{};
viewportState.sType = VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO;
viewportState.viewportCount = 1;
viewportState.scissorCount = 1;

// 5. Rasterizer
VkPipelineRasterizationStateCreateInfo rasterizer{};
rasterizer.sType = VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO;
rasterizer.depthClampEnable = VK_FALSE;
rasterizer.rasterizerDiscardEnable = VK_FALSE;
rasterizer.polygonMode = VK_POLYGON_MODE_FILL;
rasterizer.lineWidth = 1.0f;
rasterizer.cullMode = VK_CULL_MODE_NONE;
rasterizer.frontFace = VK_FRONT_FACE_CLOCKWISE;
rasterizer.depthBiasEnable = VK_FALSE;

// 6. Multisample
VkPipelineMultisampleStateCreateInfo multisampling{};
multisampling.sType = VK_STRUCTURE_TYPE_PIPELINE_MULTISAMPLE_STATE_CREATE_INFO;
multisampling.sampleShadingEnable = VK_FALSE;
multisampling.rasterizationSamples = VK_SAMPLE_COUNT_1_BIT;
```

CreateParticleCloud()

```
GeometryGenerator::MeshData GeometryGenerator::CreateParticleCloud(int numParticles) {
    MeshData meshData;
    meshData.vertices.clear();
    meshData.indices.clear();

    for (int i = 0; i < numParticles; ++i) {
        // Normalized ID (0.0 to 1.0) used for random offsets
        float z = static_cast<float>(i) / static_cast<float>(numParticles);

        uint32_t baseIndex = static_cast<uint32_t>(meshData.vertices.size());

        // Create a quad.
        // x,y = corner offsets
        // z   = particle ID
        Vertex v1; v1.pos = { -1.0f, -1.0f, z };
        Vertex v2; v1.pos = {  1.0f, -1.0f, z };
        Vertex v3; v1.pos = {  1.0f,  1.0f, z };
        Vertex v4; v1.pos = { -1.0f,  1.0f, z };

        // Push vertices (Color/Normal irrelevant for this shader, but needed for struct padding)
        meshData.vertices.push_back(Vertex{ {-1.0f, -1.0f, z}, {1,1,1}, {0,0,0}, {0,0} }); // Bottom-Left
        meshData.vertices.push_back(Vertex{ { 1.0f, -1.0f, z}, {1,1,1}, {0,0,0}, {1,0} }); // Bottom-Right
        meshData.vertices.push_back(Vertex{ { 1.0f,  1.0f, z}, {1,1,1}, {0,0,0}, {1,1} }); // Top-Right
        meshData.vertices.push_back(Vertex{ {-1.0f,  1.0f, z}, {1,1,1}, {0,0,0}, {0,1} }); // Top-Left

        // Indices (Two triangles)
        meshData.indices.push_back(baseIndex + 0);
        meshData.indices.push_back(baseIndex + 1);
        meshData.indices.push_back(baseIndex + 2);
        meshData.indices.push_back(baseIndex + 2);
        meshData.indices.push_back(baseIndex + 3);
        meshData.indices.push_back(baseIndex + 0);
    }
    return meshData;
}
```

CreateParticleCloud makes a stack of quads where the x & y represent corner offsets for billboarding, and the z component represents the normalised particle ID, used for random motion. The UBO was updated with “time”, which is updated each frame to drive the physics simulation.
A particle pipeline was created to handle the rendering of semi-transparent particles, using new particle and vertex shaders for rendering. The vertex shader uses sin/cos for cone spread logic and the fragment uses length(fragTexCoord) to render soft circles instead of square quads.

Exercise 5+:

Question:

FRESNEL EFFECT

Solution:

Shader.frag

```
void main() {
    // 1. Setup Vectors
    vec3 N = normalize(inWorldNormal);
    vec3 V = normalize(ubo.eyePosition - inWorldPos); // View Vector (Surface to Eye)

    // 2. Calculate Reflection (Mirror)
    // reflect() expects incident vector (Eye to Surface), so we use -V
    vec3 R_reflect = reflect(-V, N);
    vec3 colorReflect = texture(skySampler, R_reflect).rgb;

    // 3. Calculate Refraction (Glass)
    // Air (1.00) to Glass (1.52) ratio approx 0.65
    float ior = 1.00 / 2.00;
    vec3 R_refract = refract(-V, N, ior);
    vec3 colorRefract = texture(skySampler, R_refract).rgb;
    colorRefract *= vec3(0.9, 0.95, 1.0); // Slight tint for glass

    // 4. Calculate Fresnel Factor (The Mix Weight)
    // dot(N, V) is 1.0 when looking straight on, and 0.0 at 90 degrees (edges).
    // We want more reflection at the edges (when dot is low).
    // pow(..., 3.0) makes the "rim" effect sharper/thinner.
    float fresnel = pow(1.0 - max(dot(N, V), 0.0), 1.0);

    // 5. Blend based on Fresnel
    // Low Fresnel (Center) -> Mostly Refraction
    // High Fresnel (Edges)  -> Mostly Reflection
    vec3 finalColor = mix(colorRefract, colorReflect, fresnel);

    outColor = vec4(finalColor, 1.0);
}
```

To achieve the Fresnel effect, the fragment shader was updated to calculate both a reflection vector for mirror-like environmental sampling and a refraction vector to simulate light passing through the object with a custom purple tint. A Fresnel factor was then computed based on the dot product of the surface normal and view direction, creating a gradient that quantifies the difference between direct and grazing viewing angles. Finally, this factor was used to mix the tinted refraction and the pure reflection, resulting in a realistic material that appears transparent in the center and reflective at the edges.

Lab 8

Date: 21/11/2025

Exercise 1:

Question:

RENDER-TO-TEXTURE ON A CUBE

Solution:

```
void HelloTriangleApplication::recordCommandBuffer(VkCommandBuffer commandBuffer, uint32_t imageIndex) {
    VkCommandBufferBeginInfo beginInfo{};
    beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;

    if (vkBeginCommandBuffer(commandBuffer, &beginInfo) != VK_SUCCESS) {
        throw std::runtime_error("Failed to begin recording command buffer!");
    }

    // Pass 1: Render geometry to the off-screen texture
    recordOffscreenPass(commandBuffer);

    // Pass 2: Render the textured cube to the screen
    recordScreenPass(commandBuffer, imageIndex);

    if (vkEndCommandBuffer(commandBuffer) != VK_SUCCESS) {
        throw std::runtime_error("Failed to record command buffer!");
    }
}
```

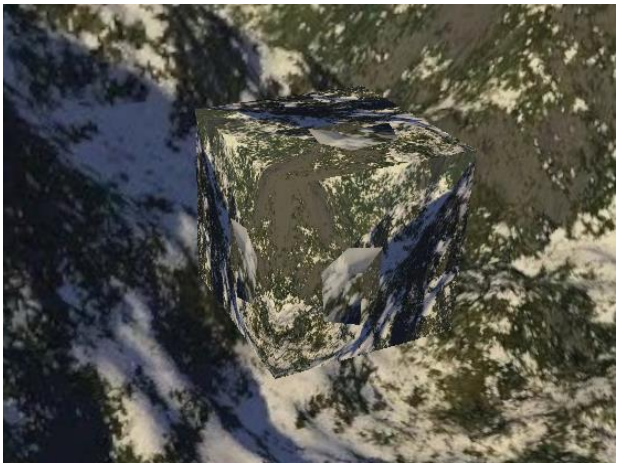
A two-pass render-to-texture system is now in place, where the offscreen pass renders a vertex-coloured 3d cube, which passes the texture to the onscreen pass, where another cube is rendered using the first pass texture.

Pass one changes from an offscreenImage to COLOR_ATTACHMENT, then to SHADER_READ_ONLY. The colour attachment is set as the swapchain image, taking the result from pass one and pasting it onto the faces of the cube. It is finally changed to PRESENT_SRC_KHR, which packages the image and hands it over to the display engine.

```
layout(location = 0) in vec2 fragTexCoord;

// Binding 1 matches the C++ descriptor setup for the offscreen image
layout(binding = 1) uniform sampler2D sceneTexture;

layout(location = 0) out vec4 outColor;

void main() {
    // Sample the texture rendered in Pass 1
    outColor = texture(sceneTexture, fragTexCoord);
}
```

```
void main() {
    mat4 view = lookAt(ubo.eyePosition, ubo.lookAtPoint, ubo.upDirection);
    mat4 proj = perspective(ubo.fovy, ubo.aspect, ubo.zNear, ubo.zFar);

    gl_Position = proj * view * pc.model * vec4(inPosition, 1.0);
    fragTexCoord = inTexCoord;
}
```

New textured cube shaders pass texture coordinates to the fragment shader and to sample the sceneTexture from the first pass to output the colour.



Exercise 2:

Question:

TEXTURE SMOOTHING (BOX BLUR)

Solution:

A blur fragment shader is created, which implements a convolution kernel to smooth the texture. The fullscreen vertex shader uses gl_vertexindex to generate a single triangle covering the entire screen, removing the need for vertex buffers in the second pass.

```
// Output texture coordinates to the fragment shader
layout (location = 0) out vec2 outUV;

void main()
{
    // Generate a triangle that covers the entire screen
    // Vertices: (-1, -1), (3, -1), (-1, 3)
    // The "outUV" calculation maps these to [0,0], [2,0], [0,2] texture space
    outUV = vec2((gl_VertexIndex << 1) & 2, gl_VertexIndex & 2);

    // Convert UV to Clip Space [-1..1]
    gl_Position = vec4(outUV * 2.0f - 1.0f, 0.0f, 1.0f);
}
```

```
void main() {
    float stepSize = 1.0;
    // Calculate size of a single texel
    vec2 texelSize = stepSize / textureSize(sceneTexture, 0);

    vec4 result = vec4(0.0);
    int boxSize = 2; // Radius of the blur (try 1, 2, 3...)

    // Convolution Loop
    // Iterates from -boxSize to +boxSize in both X and Y
    for (int x = -boxSize; x <= boxSize; x++) {
        for (int y = -boxSize; y <= boxSize; y++) {
            result += texture(sceneTexture, fragTexCoord + vec2(x, y) * texelSize);
        }
    }

    // Average the color
    // Total samples = (side width) * (side height)
    int side = (boxSize * 2 + 1);
    outColor = result / float(side * side);
}
```

The TexturedCube pipeline is modified to support 2d rendering and the screen pass is updated to remove vertex buffers in place of cmddraw.

```
// 1. Shader Modules
// Use the new shaders for the blur pass
auto vertShaderCode = readFile("shaders/fullscreen_vert.spv");
auto fragShaderCode = readFile("shaders/blur_frag.spv");

VkShaderModule vertShaderModule = createShaderModule(vertShaderCode);
VkShaderModule fragShaderModule = createShaderModule(fragShaderCode);

VkPipelineShaderStageCreateInfo shaderStages[] = {
    {VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO, nullptr, 0, VK_SHADER_STAGE_VERTEX_BIT, vertShaderModule, "main",
     nullptr},
    {VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO, nullptr, 0, VK_SHADER_STAGE_FRAGMENT_BIT, fragShaderModule,
     "main", nullptr}
};

// 2. Vertex Input State (CRUCIAL CHANGE: Empty for Full-Screen Quad)
VkPipelineVertexInputStateCreateInfo vertexInputInfo{};
vertexInputInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO;
vertexInputInfo.vertexBindingDescriptionCount = 0;   // No vertex bindings
vertexInputInfo.pVertexBindingDescriptions = nullptr;
vertexInputInfo.vertexAttributeDescriptionCount = 0; // No vertex attributes
vertexInputInfo.pVertexAttributeDescriptions = nullptr;

// 3. Input Assembly (Primitives are listed as triangles)
VkPipelineInputAssemblyStateCreateInfo inputAssembly{ VK_STRUCTURE_TYPE_PIPELINE_INPUT_ASSEMBLY_STATE_CREATE_INFO };
inputAssembly.topology = VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST;
inputAssembly.primitiveRestartEnable = VK_FALSE;

// 4. Viewport and Scissor (Dynamic)
VkPipelineViewportStateCreateInfo viewportState{ VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO };
viewportState.viewportCount = 1;
viewportState.scissorCount = 1;

// 5. Rasterization State (Disable culling since we draw a single quad)
VkPipelineRasterizationStateCreateInfo rasterizer{ VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO };
rasterizer.sType = VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO;
rasterizer.depthClampEnable = VK_FALSE;
rasterizer.rasterizerDiscardEnable = VK_FALSE;
rasterizer.polygonMode = VK_POLYGON_MODE_FILL;
rasterizer.lineWidth = 1.0f;
rasterizer.cullMode = VK_CULL_MODE_NONE; // CRUCIAL: No culling
rasterizer.frontFace = VK_FRONT_FACE_CLOCKWISE;
rasterizer.depthBiasEnable = VK_FALSE;
```

```
// 6. Multisampling (Disabled)
VkPipelineMultisampleStateCreateInfo multisampling{ VK_STRUCTURE_TYPE_PIPELINE_MULTISAMPLE_STATE_CREATE_INFO };
multisampling.rasterizationSamples = VK_SAMPLE_COUNT_1_BIT;

// 7. Depth Stencil State (CRUCIAL CHANGE: Disabled for 2D post-process)
VkPipelineDepthStencilStateCreateInfo depthStencil{ VK_STRUCTURE_TYPE_PIPELINE_DEPTH_STENCIL_STATE_CREATE_INFO };
depthStencil.depthTestEnable = VK_FALSE;
depthStencil.depthWriteEnable = VK_FALSE;
depthStencil.depthCompareOp = VK_COMPARE_OP_ALWAYS;
depthStencil.stencilTestEnable = VK_FALSE;

// 8. Color Blending (Simple replace, no blending)
VkPipelineColorBlendAttachmentState colorBlendAttachment{};
colorBlendAttachment.colorWriteMask = VK_COLOR_COMPONENT_R_BIT | VK_COLOR_COMPONENT_G_BIT | VK_COLOR_COMPONENT_B_BIT
  VK_COLOR_COMPONENT_A_BIT;
colorBlendAttachment.blendEnable = VK_FALSE;

VkPipelineColorBlendStateCreateInfo colorBlending{ VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_STATE_CREATE_INFO };
colorBlending.logicOpEnable = VK_FALSE;
colorBlending.attachmentCount = 1;
colorBlending.pAttachments = &colorBlendAttachment;

// 9. Dynamic State
std::vector<VkDynamicState> dynamicStates = { VK_DYNAMIC_STATE_VIEWPORT, VK_DYNAMIC_STATE_SCISSOR };
VkPipelineDynamicStateCreateInfo dynamicState{ VK_STRUCTURE_TYPE_PIPELINE_DYNAMIC_STATE_CREATE_INFO };
dynamicState.dynamicStateCount = static_cast<uint32_t>(dynamicStates.size());
dynamicState.pDynamicStates = dynamicStates.data();

// 10. Pipeline Layout (Reused from Exercise 1)
VkPushConstantRange pushConstantRange{}; // Used model matrix in Ex1, not strictly needed here but keep the layout
pushConstantRange.stageFlags = VK_SHADER_STAGE_VERTEX_BIT;
pushConstantRange.offset = 0;
pushConstantRange.size = sizeof(PushConstants);

VkPipelineLayoutCreateInfo pipelineLayoutInfo{};
pipelineLayoutInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO;
pipelineLayoutInfo.setLayoutCount = 1;
pipelineLayoutInfo.pSetLayouts = &texturedCubeDescriptorSetLayout;
pipelineLayoutInfo.pushConstantRangeCount = 1;
pipelineLayoutInfo.pPushConstantRanges = &pushConstantRange;

// Only create layout if it doesn't exist (e.g., in a recreate call)
if (texturedCubePipelineLayout == VK_NULL_HANDLE) {
    if (vkCreatePipelineLayout(device, &pipelineLayoutInfo, nullptr, &texturedCubePipelineLayout) != VK_SUCCESS) {
        throw std::runtime_error("failed to create blur pipeline layout!");
    }
}
```

Exercise 3:

Question:

Glow effect

Solution:

The 3D texture wrapped cube is rendered in the offscreen image. A fullscreen quad is then rendered directly to the swapchain, using the output of pass one as its input texture.

The blur fragment shader calculates the glow, by sampling neighbouring pixels and averaging them. This is then used in the fullscreen vertex shader, where the glow can be allowed to extend outside the cubes physical edges, by running the blur effect on every pixel of the screen.

```
void main() {
    // --- 1. Calculate the Blur (Same as Exercise 2) ---
    float stepSize = 1.0;
    vec2 texelSize = stepSize / textureSize(sceneTexture, 0);

    vec4 blurSum = vec4(0.0);
    int boxSize = 4; // Increased size slightly for a wider glow

    // Convolution Loop
    for (int x = -boxSize; x <= boxSize; x++) {
        for (int y = -boxSize; y <= boxSize; y++) {
            blurSum += texture(sceneTexture, fragTexCoord + vec2(x, y) * texelSize);
        }
    }
    // Average the samples to get the blurred color
    int side = (boxSize * 2 + 1);
    vec4 blurredColor = blurSum / float(side * side);

    // --- 2. Sample the Original Sharp Image ---
    vec4 originalColor = texture(sceneTexture, fragTexCoord);

    // --- 3. Additive Blending (Glow Effect) ---
    // Formula: Final = Original + Blurred
    // The blur bleeds brightness into the surrounding area.
    // When added to the original, bright spots become "over-exposed" (white)
    // and cast a halo, creating the "dreamy" glow.
    outColor = originalColor + (blurredColor * 1.2);
}
```

Additive blending is used to overlay the glow onto the skybox.

Exercise 4:

Question:

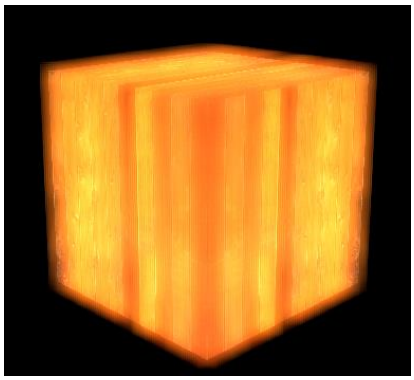
Fire animation

Solution:

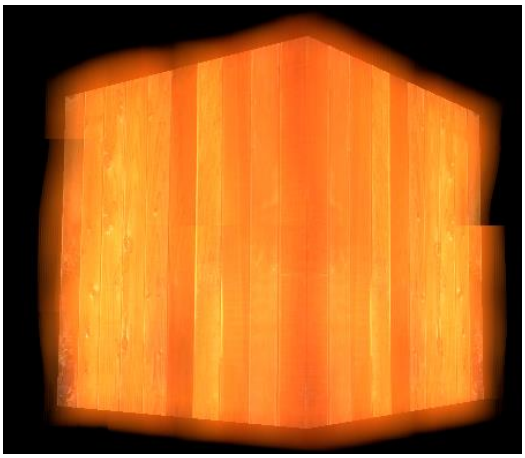
```
void main() {
    // --- 1. CONFIGURATION ---
    // RADIUS: How far the fire spreads.
    int boxSize = 4;

    // SPREAD: How far apart pixels are sampled.
    float stepSize = 2.0;

    // INTENSITY: Base brightness.
    float baseIntensity = 4.0;

    // --- 2. Calculate the Wide Blur ---
    vec2 texelSize = stepSize / textureSize(sceneTexture, 0);
    vec4 blurSum = vec4(0.0);

    for (int x = -boxSize; x <= boxSize; x++) {
        for (int y = -boxSize; y <= boxSize; y++) {
            blurSum += texture(sceneTexture, fragTexCoord + vec2(x, y) * texelSize);
        }
    }

    int side = (boxSize * 2 + 1);
    vec4 blurredColor = blurSum / float(side * side);

    // --- 3. Fire Animation ---

    // Tint: Hot Orange/Yellow (More Yellow added for brightness)
    vec4 fireColor = blurredColor * vec4(1.0, 0.6, 0.2, 1.0);

    // Flicker: More aggressive math for a raging fire
    // We combine 3 waves now for extra chaos
    float flicker = sin(ubo.time * 15.0) * 0.5   // Fast wave
                  + sin(ubo.time * 25.0) * 0.3   // Faster wave
                  + sin(ubo.time * 5.0)  * 0.2;  // Slow breath

    float totalIntensity = baseIntensity + flicker;

    // --- 4. Final Output ---
    vec4 originalColor = texture(sceneTexture, fragTexCoord);

    outColor = originalColor + (fireColor * totalIntensity);
}
```

The stage flag in the uboBinding is updated to accept vertex and fragment bit.

Blur shader uses a sine wave * time to give the cube glow a flicker, which is pared with an orange tint to give the flame effect.



True Bloom



Final Lab – The Orb

Review of Algorithms

Geometry Representation and Processing

The project produces standardised *Geometry* objects, containing indexed vertex buffers stored in array of structures format, defined in *Vertex.h*, containing Position, colour, Texture Coordinates and Normal.

Meshes can be imported as .obj files, where an unordered map is used to map unique combinations of positon/uv/normal indices to a single vertex index in the final buffer. The loader uses hashing to identify unique combinations of attributes, rather than unique positions, meaning that if two faces share a vertex with identical attributes, the vertex can be reused instead of duplicated, reducing memory usage.

The *GeometryGenerator* creates terrain using Perlin noise, using one frequency to define the base layer, giving a general shape, and another to add surface detail. To create island-like geometry, the heights are clamped to zero when the distance from the centre exceeds 95% of the radius, ensuring the terrain seamlessly merges with the top of the pedestal.

When *RecordCommandBuffer* iterates through the scene objects, it updates a *PushConstantObject* struct for every entity. The model matrix is passed via push constant, allowing the same *Geometry* buffer to be drawn multiple at positions without modifying the vertex buffer itself. Control flags like *shadingMode*, *layerMask*, and *burnFactor* are packed into the push constant block. This allows the shader to switch between Gouraud/Phong shading or apply burning effects per-object instantly, avoiding the overhead of descriptor set updates.

Shading and Lighting

The rendering engine implements a hybrid shading pipeline that supports both Gouraud and Phong shading models, with lighting calculations utilising the Blinn-Phong reflection model, where specular highlights are determined by the dot product of the normal and halfway vector. Point lights use quadratic falloff to localise the illumination, while directional lights apply no attenuation.

Refraction is achieved by sampling a captured off-screen framebuffer, where texture coordinated are modified based on the surface normal to simulate distortion. Layer masks are used to restrict light sources to specific geometry, preventing indoor illumination for affecting exterior surfaces.

Shadow Generation

Shadows are generated using a directional shadow mapping technique with an orthographic projection centred on the scene. To mitigate aliasing artifacts along shadow edges, the fragment shader implements percentage-closer filtering, which samples a 3x3 grid of texels around the current fragment and averages the result. Surface acne is precented using a dynamic slope-scaled bias, which adjusts the shadow based on the angle of the light source relative to the surface normal.

Particle System

Hardware instancing is utilised to efficiently render large numbers of particles with a single draw call. A shared quad mesh is stored in the vertex buffer, while per-instance data is supplied by a secondary vertex buffer. The simulation logic, including velocity integration and lifecycle management, is performed on the CPU. The updated instance data is then copied and mapped to a GPU buffer every frame, allowing for dynamic behaviour like size variation and colour interpolation.

Connecting Application Objects and Graphics

The *SceneObject* struct acts as a lightweight container for specific instance data, holding the logical state required to simulate the entity in the world. This includes a transform matrix for position, rotation, and scale; simulation variables such as *currentTemp*, *ObjectState*, and *isFlammable*; and instance-specific rendering instructions like *texturePath*, *shadingMode*, and *burnFactor*. The heavy graphical data is encapsulated in the *Geometry* class, which owns the vertex buffers and indices on the GPU. The link between these systems is established through a *shared_ptr<Geometry>* member within the *SceneObject*, allowing multiple logical objects to reference the same underlying mesh data.

Application behaviour is simulated entirely on the CPU within the *Scene::Update* loop, ensuring strict separation between game logic and rendering. Orbital mechanics are calculated frame-by-frame using quaternion rotation to update the transform matrix, while a thermodynamics system drives state transitions, such as modifying *currentTemp* based on heat sources or incrementing the *burnFactor* as an object chars. For quick visual changes, the engine leverages the decoupling of geometry to swap the *shared_ptr<Geometry>* to an *AshPile* prototype when an object transitions to the *BURNT* state, instantly altering its mesh without destroying the logical entity.

These CPU-side updates are propagated to the graphics pipeline using two primary methods. Continuous state changes, such as the updated transform matrix and scalar *burnFactor*, are packed into a *PushConstantObject* and sent directly to the command buffer via *vkCmdPushConstants* for every draw call, ensuring the shader receives the exact frame state. The particle system utilises memory mapping, where the simulation calculates new particle positions on the CPU and uses *memcpy* to transfer the data to a mapped GPU instance buffer immediately before drawing.

Extensions and Improvements

The main critique of this project is the lack of abstraction of game logic from the *Scene* class and *SceneObject* struct. Further decoupling the transform, rendering, physics, thermodynamics and orbital components would improve the program's modularity, and should be treating as a priority for the next stages. More use could be made of the GPU, with the use of a deferred rendering pipeline for light calculation, instead of a single shader containing multiple if statements, which introduces improved performance and the possibility of new advanced features, such as illuminating sparks. Similarly, a Compute shader could be utilised to alleviate the stress created by particle calculations and the movement of data from the CPU to GPU. The use of a cascaded shadow map would reduce VRAM usage, while providing better detail nearer the camera.

Upon decoupling the game logic from *Scene*, continuations of this project could integrate bump and displacement mapping for improved visuals, along with the support for high dynamic range rendering. The environment could be improved with the inclusion wind, which would impact dust, fire spread and weather. An audio engine would also make an excellent addition to the program.